



VentaGamer

Plataforma web de venta de productos gaming

Documentación Técnica — Trabajo Práctico Integrador

Universidad: Universidad Abierta Interamericana
(UAI)

Materia: Desarrollo y Arquitecturas Web

Profesor: Escandell Gustavo Emanuel

Integrantes:
Fazzari, Franco Tomás
Mastromarino, Nicolás
Reser, Iván Leonel

Fecha de entrega: 09/07/2026

Índice y Carátula

VentaGamer — Documentación técnica

Carátula

Campo	Dato
Universidad	Universidad Abierta Interamericana (UAI)
Materia	Desarrollo y Arquitecturas Web
Trabajo	Trabajo Práctico Integrador
Sistema	VentaGamer — Plataforma web de venta de productos gaming
Profesor/a	Escandell Gustavo Emanuel
Integrantes	Fazzari, Franco Tomás · Mastromarino, Nicolás · Reser, Iván Leonel
Fecha de entrega	09/07/2026

Versión PDF: los documentos también están disponibles en la carpeta `PDFs/` (generados con `pdf-build/generar-pdfs.mjs`).

Propósito de esta carpeta

Esta carpeta reúne la **documentación técnica de entrega** del Trabajo Práctico Integrador. Está pensada para que cualquier lector (docente o estudiante) pueda comprender qué problema resuelve el sistema, cómo está construido y cómo ponerlo en funcionamiento, sin necesidad de leer previamente el código fuente.

La documentación está dividida en varios documentos Markdown para facilitar su lectura y edición. Todos los diagramas están escritos en **Mermaid**, por lo que pueden editarse como texto y se renderizan automáticamente en GitHub, VS Code y la mayoría de los visores de Markdown.

La carpeta `docs/` del repositorio contiene material interno de desarrollo y no forma parte de esta entrega.

Índice de documentos

#	Documento	Contenido
1	Problema y temática	Definición del problema, justificación, objetivo general y público objetivo
2	Requerimientos	Actores del sistema, requerimientos funcionales y no funcionales
3	Casos de uso	Diagrama general y casos de uso detallados
4	Arquitectura	Arquitectura de despliegue (Docker), backend, frontend y stack tecnológico
5	Base de datos	DER, detalle de tablas, migraciones EF Core y script SQL demostrativo
6	APIs	Referencia de la API REST, autenticación JWT, manejo de errores y SignalR
7	Frontend	Páginas, manipulación del DOM, diseño responsivo, formularios y multi-idioma
8	Calidad y normativa	Buenas prácticas, seguridad, usabilidad, accesibilidad y Ley 25.326
9	Chatbot con IA	Asistente "GG": arquitectura, herramientas y streaming
10	Backup y recuperación	Estrategia de respaldo, integridad HMAC y procedimiento de recuperación
11	Ejecución del proyecto	Cómo levantar el sistema con Docker, usuarios demo y solución de problemas
—	sql/ventagamer - demo.sql	Script SQL simplificado con fines demostrativos

Correspondencia con la rúbrica de evaluación

La siguiente tabla mapea cada semana de la rúbrica del trabajo práctico con el documento que cubre lo evaluado.

Semana	Qué se evalúa	Documento que lo cubre
1	Claridad y profundidad del problema, coherencia de la temática, justificación	01 — Problema y temática
2	Requerimientos funcionales (≥ 10) y no funcionales (≥ 5), redacción técnica	02 — Requerimientos
3	Actores, casos de uso completos (≥ 5), arquitectura cliente-servidor, módulos	03 — Casos de uso · 04 — Arquitectura
4	Entidades, DER ($\geq 4-6$ tablas), claves primarias y foráneas, script SQL	05 — Base de datos · sql/
5	Estructura HTML, navegación interna, estilos CSS, organización de la interfaz	07 — Frontend
6	Diseño responsivo, manipulación del DOM, eventos, formularios y validaciones	07 — Frontend
7	Integración de API con fetch, datos dinámicos, manejo de errores	06 — APIs
8	Persistencia de datos, consultas, filtros y búsquedas	05 — Base de datos · 06 — APIs
9	Calidad del código, cumplimiento normativo (Ley 25.326), usabilidad, extras	08 — Calidad y normativa · 09 — Chatbot · 10 — Backup
10	Funcionamiento integral, explicación técnica, dominio del sistema	Toda la carpeta · 11 — Ejecución

Guía de lectura sugerida

1. Comenzar por el documento **01** para entender el problema y el alcance del sistema.
2. Continuar con **02** y **03** para conocer qué hace el sistema y quiénes lo usan.
3. Leer **04** y **05** para comprender cómo está construido (arquitectura y datos).
4. Consultar **06** y **07** para el detalle de API y frontend.
5. Cerrar con **08**, **09** y **10** (calidad, normativa y funcionalidades avanzadas).
6. El documento **11** permite ejecutar el sistema y verificar todo lo anterior en funcionamiento.

Problema y Temática

1. Problema y temática

1.1. Temática elegida

La temática elegida es el **comercio electrónico de productos gaming**: consolas, periféricos (mandos, teclados, mouse, auriculares), monitores, sillas, equipamiento de streaming y videojuegos.

El sistema desarrollado se llama **VentaGamer** y es una plataforma web de venta online que integra catálogo de productos, carrito de compras, gestión de pedidos con comprobantes en PDF, administración de usuarios con roles y permisos, auditoría de operaciones, respaldo de datos y un asistente virtual con inteligencia artificial.

1.2. Definición del problema

¿Qué problema se resuelve?

Los comercios dedicados a la venta de productos gaming suelen gestionar su operación con herramientas dispersas: planillas de cálculo para el stock, redes sociales para mostrar productos y mensajería para concretar ventas. Este esquema presenta problemas concretos:

- **Falta de un canal de venta unificado.** El cliente no puede ver el catálogo completo, consultar stock real ni comprar de forma autónoma; cada venta requiere intervención manual.
- **Ausencia de control de stock confiable.** Al no existir una fuente única de datos, se venden productos sin stock o se pierden ventas por desconocimiento de la disponibilidad real.
- **Sin trazabilidad de operaciones.** No queda registro de quién modificó un precio, quién dio de baja un producto o cuándo se realizó una venta, lo que dificulta detectar errores y deslinda responsabilidades.
- **Datos sin resguardo.** La información comercial (clientes, ventas, stock) no cuenta con mecanismos de respaldo ni de verificación de integridad, con el riesgo de pérdida total ante una falla.
- **Atención al cliente limitada.** Las consultas repetitivas (precios, stock, estado de un pedido) consumen tiempo del personal y demoran la respuesta al cliente.

¿A quién afecta?

- A los **clientes**, que no disponen de un medio ágil y autónomo para explorar productos, comparar precios y comprar.

- A los **administradores del comercio**, que carecen de herramientas centralizadas para gestionar catálogo, usuarios y ventas.
- Al **personal técnico**, que no cuenta con mecanismos de auditoría, respaldo y recuperación de la información.

¿Por qué es importante resolverlo?

El comercio electrónico es hoy el canal de venta dominante en el rubro tecnológico. Un comercio sin plataforma propia depende de terceros, pierde competitividad y expone sus datos a riesgos evitables. Resolver este problema con una aplicación web integral permite: vender de forma autónoma las 24 horas, mantener un stock consistente, cumplir con la normativa de protección de datos personales y garantizar la continuidad del negocio ante fallas.

1.3. Justificación de la elección

Se eligió esta temática por tres razones:

1. **Riqueza funcional.** Una tienda online exige resolver problemas variados y representativos de la ingeniería de software: autenticación y autorización, transacciones (una compra debe descontar stock y registrar el pedido de forma atómica), generación de documentos, auditoría, respaldo e internacionalización. Esto permite aplicar de manera natural todos los contenidos de la materia.
2. **Cercanía con el usuario final.** El dominio es conocido por cualquier persona (todos compramos online), lo que facilita validar los requerimientos y evaluar la usabilidad del resultado.
3. **Escalabilidad del planteo.** El modelo de datos y la arquitectura elegida permiten crecer hacia escenarios de mayor volumen (más productos, más usuarios, más ventas) sin rediseñar el sistema.

1.4. Nombre del sistema

VentaGamer — plataforma web de venta de productos gaming.

1.5. Objetivo general

Desarrollar una **aplicación web responsiva** de comercio electrónico que permita a los clientes explorar un catálogo de productos gaming, gestionar un carrito y concretar compras con comprobante descargable; y a los perfiles administrativos, gestionar el catálogo, los usuarios, los roles y permisos, la auditoría de operaciones y el respaldo de la información — todo ello sobre una arquitectura cliente-servidor con API REST, base de datos relacional y estándares de calidad, seguridad y cumplimiento normativo (Ley 25.326 de Protección de Datos Personales).

1.6. Objetivos específicos

- Ofrecer un catálogo navegable con búsqueda, filtrado por categoría y paginación.

- Implementar registro e inicio de sesión seguros, con bloqueo automático ante intentos fallidos reiterados.
- Permitir la compra mediante carrito y checkout transaccional que garantice la consistencia del stock.
- Emitir comprobantes de compra en formato PDF.
- Administrar usuarios, roles y permisos con un esquema de autorización granular.
- Registrar automáticamente en una bitácora las operaciones relevantes del sistema.
- Soportar múltiples idiomas (español, inglés y portugués).
- Proveer mecanismos de respaldo de la base de datos y verificación de integridad de los datos.
- Incorporar un asistente virtual con IA que ayude a clientes y administradores a consultar información del sistema en lenguaje natural.

1.7. Público objetivo

Perfil	Descripción
Clientes	Personas interesadas en productos gaming que desean explorar el catálogo y comprar online. Incluye visitantes no registrados (solo consulta) y usuarios registrados (compra).
Administradores	Personal del comercio responsable del catálogo de productos, de los usuarios y de la configuración del sistema.
WebMasters	Personal técnico responsable del mantenimiento: auditoría, copias de seguridad y verificación de integridad.
Testers	Perfil de control de calidad con acceso mixto de solo consulta para verificar el funcionamiento del sistema.

1.8. Alcance

Incluye: catálogo con búsqueda y filtros, registro/login con JWT, carrito y checkout transaccional, comprobantes PDF, panel de administración (productos, usuarios, roles y permisos), bitácora de auditoría, backups de base de datos, verificación de integridad HMAC, multi-idioma, chatbot con IA (Ollama) y despliegue completo con Docker.

No incluye (fuera de alcance de esta versión): pasarela de pagos real, gestión de envíos y logística, notificaciones por correo electrónico, y recuperación de contraseña operativa (la interfaz existe a modo demostrativo, sin backend asociado).

Requerimientos

2. Actores y requerimientos

2.1. Actores del sistema

Un **actor** es cualquier entidad externa que interactúa con el sistema. En VentaGamer los actores humanos se corresponden con los roles definidos en la base de datos (tabla **Roles**), a los que se suma el visitante no autenticado y dos actores de sistema.

Actor	Tipo	Descripción
Visitante	Humano	Persona no autenticada. Puede navegar el catálogo, buscar y filtrar productos, registrarse e iniciar sesión. No puede comprar.
Cliente (rol User)	Humano	Usuario registrado. Además de lo anterior, gestiona su carrito, realiza compras, consulta sus pedidos y descarga comprobantes PDF. Puede conversar con el asistente IA.
Administrador (rol Admin)	Humano	Gestiona el catálogo de productos (altas, bajas y modificaciones), los usuarios (bloqueo, cambio de rol), los roles y permisos, y la configuración del asistente IA. No realiza compras.
WebMaster (rol WebMaster)	Humano	Perfil técnico. Consulta la bitácora de auditoría, genera copias de seguridad y ejecuta verificaciones de integridad de los datos.
Tester (rol Tester)	Humano	Perfil de control de calidad con acceso mixto de consulta: catálogo, carrito, roles y bitácora.
Asistente GG	Sistema (IA)	Chatbot que responde consultas en lenguaje natural. Actúa en nombre del usuario autenticado: solo accede a los datos que los permisos de ese usuario habilitan.
Servidor Ollama	Sistema externo	Motor de inferencia del modelo de lenguaje que da soporte al asistente GG.

Permisos por rol

La autorización es granular: cada rol posee un conjunto de **permisos** (12 en total) y las pantallas y endpoints exigen permisos puntuales, no roles. El detalle completo se desarrolla en **06 — APIs** y **08 — Calidad y normativa**.

Permiso	Cliente (User)	Admin	WebMaster	Tester
<code>products.read</code> — ver catálogo	✓	✓	—	✓
<code>products.write</code> — ABM de productos	—	✓	—	—
<code>cart.use</code> — carrito y compra	✓	—	—	✓
<code>orders.read.own</code> — ver compras propias	✓	—	—	—
<code>orders.read.all</code> — ver todas las compras	—	✓	—	—
<code>users.register</code> — gestionar usuarios	—	✓	—	—
<code>roles.read</code> — ver roles y permisos	—	✓	—	✓
<code>roles.write</code> — ABM de roles	—	✓	—	—
<code>audit.read</code> — consultar bitácora	—	✓	✓	✓
<code>backup.manage</code> — crear/listar backups	—	✓	✓	—
<code>integrity.check</code> — verificar integridad	—	✓	✓	—
<code>config.read</code> — ver configuración	✓	✓	✓	✓

El rol Admin recibe por seed todos los permisos excepto los de compra (`cart.use`, `orders.read.own`), reflejando que el administrador gestiona el sistema pero no compra. Además, el modelo soporta **jerarquía de roles**: un rol puede heredar los permisos de otros roles "padre" (tabla `RoleHierarchies`).

2.2. Requerimientos funcionales

ID	Requerimiento	Módulo
RF-01	El sistema debe permitir el registro de nuevos usuarios con nombre de usuario único, contraseña de al menos 8 caracteres e idioma preferido. Ante un nombre ya existente debe informarse el conflicto.	Autenticación
RF-02	El sistema debe permitir el inicio de sesión con usuario y contraseña, devolviendo un token JWT con vigencia de 60 minutos que habilita las operaciones protegidas.	Autenticación
RF-03	El sistema debe bloquear automáticamente una cuenta tras 3 intentos consecutivos de inicio de sesión fallidos, e informar la situación al usuario. Solo un administrador puede desbloquearla.	Autenticación
RF-04	El sistema debe exhibir un catálogo de productos paginado , con búsqueda por texto libre y filtrado por categoría, accesible sin necesidad de autenticarse.	Catálogo
RF-05	El sistema debe permitir a los usuarios con permiso de compra gestionar un carrito : agregar productos, modificar cantidades y quitar ítems, validando en todo momento el stock disponible.	Carrito
RF-06	El sistema debe permitir confirmar la compra (checkout) de forma transaccional: verificar stock, descontarlo, generar el pedido con número único (formato VG-fecha-sufijo) y vaciar el carrito en una única operación atómica.	Pedidos
RF-07	El sistema debe permitir al cliente consultar sus pedidos y descargar el comprobante en PDF de cada uno; los usuarios con permiso ampliado pueden consultar los pedidos de todos los clientes.	Pedidos
RF-08	El sistema debe permitir a los administradores realizar el ABM de productos (alta, modificación y baja lógica), incluyendo título, categoría, precio, stock e imagen.	Administración
RF-09	El sistema debe permitir a los administradores gestionar usuarios : listar, bloquear/desbloquear y cambiar el rol asignado.	Administración
RF-10	El sistema debe permitir el ABM de roles y la asignación de permisos , incluyendo la posibilidad de que un rol herede los permisos de otros (jerarquía de roles).	Administración
RF-11	El sistema debe registrar automáticamente en una bitácora las operaciones de creación, modificación y eliminación sobre productos, usuarios, roles, permisos y pedidos, indicando fecha, módulo, usuario y detalle. La bitácora debe poder consultarse con filtros por usuario, módulo y rango de fechas.	Auditoría
RF-12	El sistema debe permitir generar copias de seguridad de la base de datos y consultar el historial de backups realizados.	Mantenimiento
RF-13	El sistema debe ofrecer una verificación de integridad de los datos críticos (usuarios y pedidos) mediante firmas HMAC-SHA256.	Mantenimiento
RF-14	El sistema debe ofrecer su interfaz en tres idiomas (español, inglés y portugués), con las traducciones almacenadas en la base de datos y seleccionables por el	Internacionalización

ID	Requerimiento	Módulo
	usuario.	
RF-15	El sistema debe ofrecer un asistente virtual con IA ("GG") capaz de responder consultas en lenguaje natural sobre catálogo, stock y pedidos, respetando los permisos del usuario que consulta y transmitiendo las respuestas en tiempo real (streaming).	Chatbot IA

2.3. Requerimientos no funcionales

ID	Requerimiento	Categoría
RNF-01	Diseño responsivo: la interfaz debe adaptarse a móviles, tablets y escritorio, con navegación utilizable en todos los tamaños de pantalla (menú colapsable en móvil, grillas adaptativas).	Usabilidad
RNF-02	Seguridad de credenciales: las contraseñas deben almacenarse con hash PBKDF2 (nunca en texto plano) y la autenticación de la API debe realizarse mediante tokens JWT firmados con HMAC-SHA256.	Seguridad
RNF-03	Autorización granular: el acceso a cada operación protegida debe validarse en el servidor mediante políticas basadas en permisos, independientemente de lo que la interfaz muestre u oculte.	Seguridad
RNF-04	Protección contra abuso: la API debe limitar la tasa de solicitudes (60 solicitudes por minuto por IP en general, y 5 por minuto en los endpoints de login/registro) para mitigar ataques de fuerza bruta y denegación de servicio.	Seguridad
RNF-05	Consistencia transaccional: las operaciones de compra deben ejecutarse dentro de una transacción de base de datos; ante cualquier falla, el sistema debe revertir todos los cambios (no puede descontarse stock sin registrarse el pedido, ni viceversa).	Confiabilidad
RNF-06	Consultas eficientes: las búsquedas y listados deben resolverse con paginación e índices de base de datos (índices sobre categoría, título, número de pedido, fechas de bitácora, etc.) para mantener tiempos de respuesta adecuados al crecer el volumen de datos.	Rendimiento
RNF-07	Mantenibilidad: el backend debe organizarse en capas con dependencias unidireccionales (Clean Architecture) y el frontend en módulos por funcionalidad, con tipado estático (TypeScript / C#) y verificación automática de estilo (ESLint).	Mantenibilidad
RNF-08	Portabilidad: todo el sistema debe poder ejecutarse en cualquier equipo con Docker mediante un único comando de orquestación (<code>docker compose up</code>), sin instalación manual de dependencias.	Portabilidad
RNF-09	Cumplimiento normativo: el tratamiento de datos personales debe ajustarse a la Ley 25.326 de Protección de Datos Personales de la República Argentina, minimizando los datos recolectados e informando su uso.	Normativa
RNF-10	Recuperabilidad: el sistema debe proveer mecanismos de respaldo de la información y un procedimiento documentado de recuperación ante fallos.	Confiabilidad

2.4. Trazabilidad requerimientos → módulos del sistema

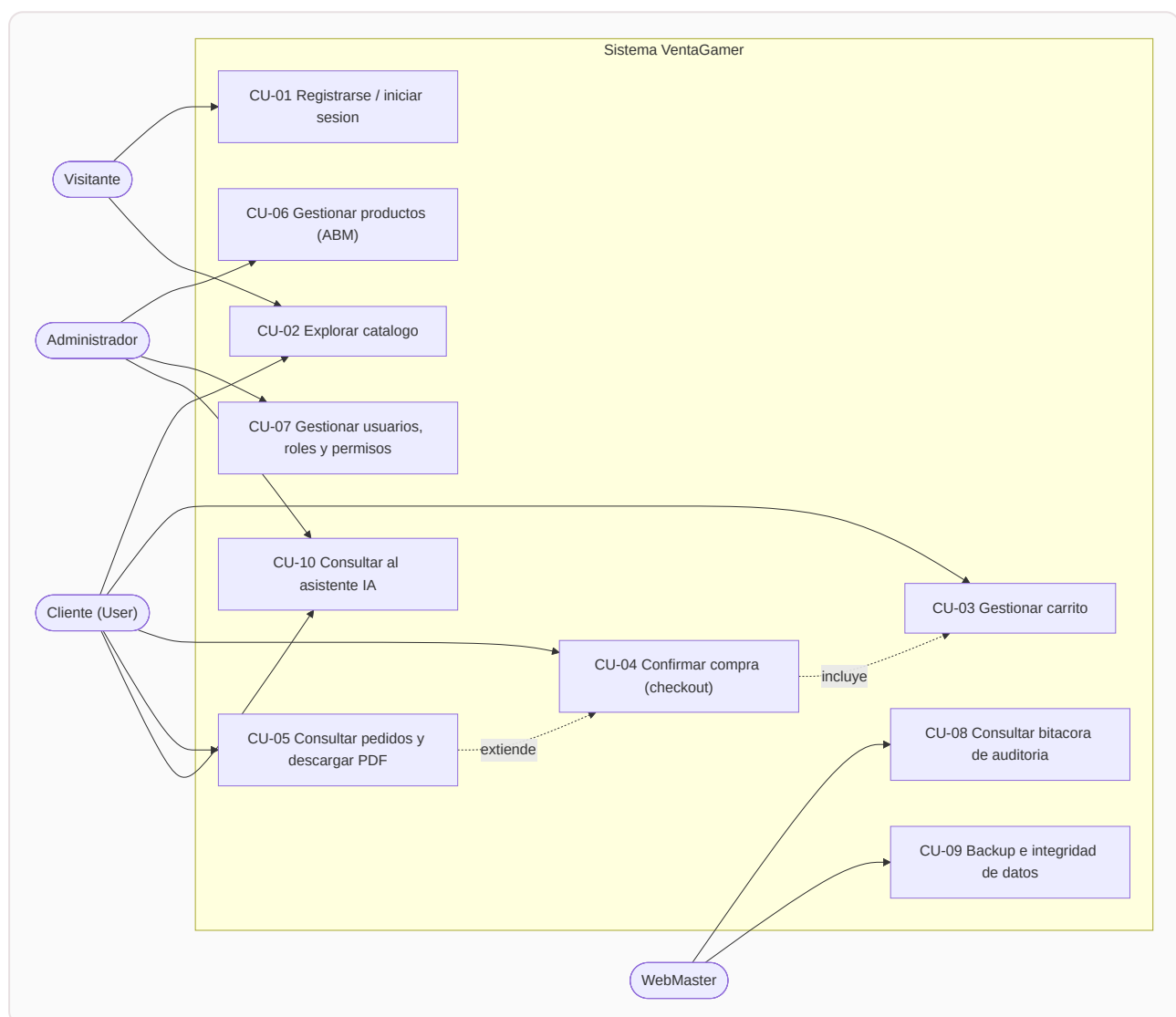
Módulo	Requerimientos	Dónde está implementado
Autenticación	RF-01, RF-02, RF-03, RNF-02	backend/src/VentaGamer.Infrastructure/Auth/ · frontend/src/features/auth/
Catálogo	RF-04, RNF-06	backend/.../Products/ · frontend/src/routes/CatalogPage.tsx
Carrito	RF-05	backend/.../Carts/ · frontend/src/routes/CartPage.tsx
Pedidos	RF-06, RF-07, RNF-05	backend/.../Orders/ · frontend/src/routes/MyOrdersPage.tsx
Administración	RF-08, RF-09, RF-10, RNF-03	backend/.../Admin/ · frontend/src/routes/AdminPage.tsx
Auditoría	RF-11	backend/.../Audit/ (interceptor de EF Core) · frontend/src/routes/AuditPage.tsx
Mantenimiento	RF-12, RF-13, RNF-10	backend/.../Maintenance/ · frontend/src/routes/MaintenancePage.tsx
Internacionalización	RF-14	backend/.../I18nController.cs · frontend/src/lib/i18n.ts
Chatbot IA	RF-15	backend/.../Ai/ · frontend/src/features/ai/
Transversal	RNF-01, RNF-04, RNF-07, RNF-08, RNF-09	Tailwind CSS · rate limiting en Program.cs · docker-compose.yml

Casos de Uso

3. Casos de uso

Este documento describe los casos de uso principales de VentaGamer. Cada uno se presenta con formato completo: actores, objetivo, precondiciones, postcondiciones, escenario principal y escenarios alternativos o de excepción.

3.1. Diagrama general de casos de uso



El asistente IA (CU-10) se describe en detalle en el documento 09 — Chatbot con IA; aquí se incluye su ficha resumida.

CU-01 — Registrarse e iniciar sesión

Campo	Detalle
Actores	Visitante
Objetivo	Obtener una cuenta y una sesión autenticada que habilite las operaciones protegidas del sistema.
Precondiciones	El visitante no posee sesión activa. Para el registro, el nombre de usuario elegido no debe existir.
Postcondiciones	El usuario queda registrado (si corresponde) y autenticado: el navegador almacena un token JWT válido por 60 minutos con sus permisos.
Prioridad	Alta — habilita el resto de los casos de uso.

Escenario principal (inicio de sesión):

1. El visitante accede a la pantalla de inicio de sesión (`/login`).
2. Ingresa nombre de usuario y contraseña, y envía el formulario.
3. El sistema valida las credenciales contra el hash PBKDF2 almacenado.
4. El sistema calcula los permisos efectivos del rol del usuario (incluyendo los heredados por jerarquía de roles).
5. El sistema emite un token JWT con los datos y permisos del usuario, y registra la fecha del acceso.
6. El navegador guarda el token y redirige al catálogo. El menú se adapta a los permisos del usuario.

Escenario alternativo A — Registro de cuenta nueva:

1. El visitante accede a `/register`, completa usuario, contraseña (mínimo 8 caracteres, con confirmación), idioma preferido y acepta los términos.
2. El sistema verifica que el nombre de usuario no exista; si existe, informa el conflicto (HTTP 409) y permite corregir.
3. El sistema crea la cuenta con rol Cliente (`User`), y continúa como en el escenario principal desde el paso 5.

Escenarios de excepción:

- **E1 — Credenciales inválidas:** el sistema responde HTTP 401, muestra un mensaje genérico y aumenta el contador de intentos fallidos del usuario.
- **E2 — Cuenta bloqueada:** al tercer intento fallido consecutivo la cuenta se bloquea. Los intentos posteriores reciben HTTP 423 y el mensaje "cuenta bloqueada"; solo un administrador puede desbloquearla (ver CU-07).
- **E3 — Límite de solicitudes:** más de 5 intentos por minuto desde la misma IP son rechazados con HTTP 429 (protección contra fuerza bruta).

CU-02 — Explorar el catálogo de productos

Campo	Detalle
Actores	Visitante, Cliente, Tester
Objetivo	Encontrar productos de interés mediante navegación, búsqueda y filtros.
Precondiciones	Ninguna: el catálogo es público.
Postcondiciones	No modifica el estado del sistema (solo consulta).
Prioridad	Alta.

Escenario principal:

1. El actor accede a la página principal (/), que muestra la primera página del catálogo (productos activos).
2. El sistema presenta cada producto con imagen, título, categoría, precio y stock disponible.
3. El actor puede: escribir un texto de búsqueda (filtra por título), elegir una categoría (Consolas, Mandos, Auriculares, Mouse, Teclados, Monitores, Sillas, Streaming, Juegos) o avanzar/retroceder páginas.
4. Ante cada acción, el frontend consulta `GET /api/products` con los parámetros correspondientes y actualiza dinámicamente la grilla, sin recargar la página.

Escenarios alternativos:

- **A1 — Sin resultados:** si ningún producto coincide, se muestra un estado vacío con un mensaje claro en lugar de una grilla en blanco.
- **A2 — Ampliar imagen:** el actor puede abrir la imagen del producto en un visor (lightbox) y cerrarlo con la tecla Escape.

Escenarios de excepción:

- **E1 — API no disponible:** se muestra un mensaje de error de carga; el usuario puede reintentar.

CU-03 — Gestionar el carrito de compras

Campo	Detalle
Actores	Cliente (permiso <code>cart.use</code>)
Objetivo	Armar y ajustar la selección de productos previa a la compra.
Precondiciones	Sesión iniciada con permiso <code>cart.use</code> . Los productos a agregar deben estar activos y con stock.
Postcondiciones	El carrito del usuario refleja los ítems y cantidades elegidos (persistido en la base de datos, un carrito por usuario).
Prioridad	Alta.

Escenario principal:

1. Desde el catálogo, el cliente presiona "Agregar al carrito" en un producto.
2. El sistema crea el carrito del usuario si aún no existe (creación diferida) y agrega el ítem con cantidad 1; si el producto ya estaba en el carrito, incrementa la cantidad.
3. El contador del carrito en la barra de navegación se actualiza al instante.
4. En la página del carrito (`/cart`), el cliente puede aumentar o disminuir cantidades y quitar ítems.
5. El sistema recalcula el total en cada cambio y muestra el detalle por línea (precio unitario × cantidad).

Escenarios de excepción:

- **E1 — Stock insuficiente:** si la cantidad solicitada supera el stock, el sistema rechaza la operación e informa el stock disponible.
- **E2 — Sesión expirada:** si el token venció, la API responde HTTP 401 y el frontend cierra la sesión automáticamente, solicitando volver a ingresar.

CU-04 — Confirmar la compra (checkout)

Campo	Detalle
Actores	Cliente (permiso <code>cart.use</code>)
Objetivo	Convertir el carrito en un pedido confirmado, garantizando la consistencia del stock.
Precondiciones	Sesión iniciada; el carrito contiene al menos un ítem.
Postcondiciones	Se crea un pedido con número único; el stock de cada producto queda descontado; el carrito queda vacío. Todo ocurre de forma atómica.
Prioridad	Alta — es la transacción central del negocio.

Escenario principal:

1. En la página del carrito, el cliente presiona "Confirmar compra".
2. El sistema inicia una **transacción de base de datos** y, dentro de ella: a. Verifica el stock actual de cada producto del carrito. b. Descuenta las cantidades compradas. c. Crea el pedido (`Order`) con un número único con formato `VG-{fecha}-{sufijo}` y una **copia inmutable** de cada producto (título y precio al momento de la compra, en `OrderItem`). d. Vacía el carrito.
3. El sistema confirma la transacción y registra el evento en la bitácora.
4. El frontend redirige al detalle del pedido creado, desde donde puede descargarse el comprobante PDF (CU-05).

Escenarios de excepción:

- **E1 — Stock modificado entre medio:** si otro cliente compró el mismo producto y el stock ya no alcanza, la transacción **se revierte completa** (no se descuenta nada ni se crea el pedido) y se informa qué producto quedó sin stock.
- **E2 — Falla de la base de datos:** cualquier error dentro de la transacción provoca la reversión total; el carrito queda intacto.

Nota de diseño: guardar en `OrderItem` el título y el precio del producto al momento de la compra (patrón *snapshot*) garantiza que los pedidos históricos no cambien si el administrador modifica el catálogo después.

CU-05 — Consultar pedidos y descargar comprobante PDF

Campo	Detalle
Actores	Cliente (pedidos propios, permiso <code>orders.read.own</code>); Administrador (todos los pedidos, permiso <code>orders.read.all</code>)
Objetivo	Consultar el historial de compras y obtener el comprobante de cada una en PDF.
Precondiciones	Sesión iniciada. Existe al menos un pedido (propio, o de cualquier usuario para el caso del administrador).
Postcondiciones	No modifica el estado del sistema.
Prioridad	Media.

Escenario principal (cliente):

1. El cliente accede a "Mis compras" (`/orders`).
2. El sistema lista sus pedidos con número, fecha y total.
3. El cliente abre un pedido y ve el detalle: ítems, cantidades, precios al momento de la compra y total.
4. El cliente presiona "Descargar comprobante PDF"; el servidor genera el documento con QuestPDF (datos del pedido y tabla de ítems) y el navegador lo descarga.

Escenario alternativo A — Vista del administrador:

1. El administrador accede a `/admin/orders`, donde ve **todos** los pedidos del sistema, con posibilidad de filtrar por nombre de usuario e indicadores generales (cantidad de pedidos, facturación).

Escenarios de excepción:

- **E1 — Acceso a pedido ajeno:** si un cliente intenta acceder por URL al pedido de otro usuario, el sistema lo rechaza (solo el dueño o quien tenga `orders.read.all` puede verlo).

CU-06 — Gestionar productos (ABM)

Campo	Detalle
Actores	Administrador (permiso <code>products.write</code>)
Objetivo	Mantener actualizado el catálogo: altas, modificaciones y bajas de productos.
Precondiciones	Sesión iniciada con permiso <code>products.write</code> .
Postcondiciones	El catálogo refleja los cambios; cada operación queda registrada en la bitácora de auditoría.
Prioridad	Alta.

Escenario principal (alta):

1. El administrador accede a `/admin/products` y presiona "Nuevo producto".
2. Completa el formulario: título, categoría, precio, stock e imagen (URL).
3. El sistema valida los datos (precio y stock no negativos, campos obligatorios) y crea el producto.
4. El interceptor de auditoría registra automáticamente el alta en la bitácora, con el usuario responsable.

Escenarios alternativos:

- **A1 — Modificación:** el administrador edita un producto existente; el sistema guarda los cambios y audita qué campos se modificaron.
- **A2 — Baja lógica:** el administrador elimina un producto; el sistema **no lo borra físicamente** sino que lo desactiva (`IsActive = false`), de modo que desaparece del catálogo pero los pedidos históricos que lo referencian permanecen íntegros.
- **A3 — Panel de indicadores:** la pantalla muestra totales de productos, productos con stock bajo y sin stock, y permite filtrar por búsqueda y categoría.

Escenarios de excepción:

- **E1 — Usuario sin permiso:** si un usuario sin `products.write` invoca los endpoints de escritura, la API responde HTTP 403 aunque haya manipulado la interfaz.

CU-07 — Gestionar usuarios, roles y permisos

Campo	Detalle
Actores	Administrador (permisos <code>users.register</code> , <code>roles.read</code> , <code>roles.write</code>)
Objetivo	Administrar quién accede al sistema y con qué permisos.
Precondiciones	Sesión iniciada con los permisos correspondientes.
Postcondiciones	Los cambios de roles/permisos impactan en los próximos inicios de sesión de los usuarios afectados. Todo cambio queda auditado.
Prioridad	Media.

Escenario principal (gestión de usuarios):

1. El administrador accede a `/admin` , pestaña "Usuarios".
2. El sistema lista los usuarios con su rol, estado (activo/bloqueado) y último acceso.
3. El administrador puede **bloquear o desbloquear** una cuenta (por ejemplo, para liberar una cuenta bloqueada por intentos fallidos — ver CU-01 E2) o **cambiarle el rol**.

Escenario alternativo A — Gestión de roles:

1. En la pestaña "Roles", el administrador ve cada rol con sus permisos directos y sus roles padre.
2. Puede crear un rol nuevo, seleccionando permisos del catálogo (12 permisos) y, opcionalmente, roles padre de los que **heredará** permisos (patrón Composite sobre la tabla `RoleHierarchies`).
3. Puede editar o eliminar roles existentes.

Escenarios de excepción:

- **E1 — Rol en uso:** el sistema impide eliminar un rol que tenga usuarios asignados (HTTP 409), para no dejar cuentas sin rol.

CU-08 — Consultar la bitácora de auditoría

Campo	Detalle
Actores	WebMaster, Administrador, Tester (permiso <code>audit.read</code>)
Objetivo	Revisar la trazabilidad de las operaciones del sistema: qué se hizo, quién y cuándo.
Precondiciones	Sesión iniciada con permiso <code>audit.read</code> .
Postcondiciones	No modifica el estado del sistema.
Prioridad	Media.

Escenario principal:

1. El actor accede a `/audit`.
2. El sistema muestra la bitácora paginada, ordenada por fecha descendente, con: fecha y hora, módulo, usuario y descripción del evento (por ejemplo, `UPDATE Product(5) fields=Price,Stock`).
3. El actor puede filtrar por nombre de usuario, módulo y rango de fechas.

Nota de diseño: la bitácora se alimenta de forma **automática** mediante un interceptor de Entity Framework Core (`AuditSaveChangesInterceptor`) que detecta las inserciones, modificaciones y eliminaciones sobre las entidades relevantes en el mismo momento en que se guardan, sin que cada módulo deba acordarse de registrar el evento. Esto garantiza que ninguna operación quede sin auditar.

CU-09 — Generar backups y verificar integridad

Campo	Detalle
Actores	WebMaster, Administrador (permisos <code>backup.manage</code> , <code>integrity.check</code>)
Objetivo	Resguardar la información del sistema y verificar que los datos críticos no fueron alterados.
Precondiciones	Sesión iniciada con los permisos correspondientes; el motor SQL Server debe estar operativo.
Postcondiciones	Se genera un archivo de respaldo <code>.bak</code> en el servidor de base de datos; el historial de backups se actualiza.
Prioridad	Media.

Escenario principal (backup):

1. El actor accede a `/maintenance` y presiona "Crear backup".
2. El sistema ejecuta un `BACKUP DATABASE` comprimido hacia el volumen de datos de SQL Server.
3. El sistema muestra el historial de backups (fecha, tamaño, archivo) consultando el catálogo del motor (`msdb`).

Escenario alternativo A — Verificación de integridad:

1. El actor ejecuta la verificación de integridad.
2. El sistema calcula firmas HMAC-SHA256 sobre los datos críticos (usuarios y pedidos) y presenta un reporte con los resultados.

Escenarios de excepción:

- **E1 — Motor no disponible:** si SQL Server no responde, se informa el error sin afectar el resto del sistema.

El detalle completo de la estrategia de respaldo y del procedimiento de recuperación se desarrolla en [10 — Backup y recuperación](#).

CU-10 — Consultar al asistente IA (resumen)

Campo	Detalle
Actores	Cualquier usuario autenticado; Servidor Ollama (actor de sistema)
Objetivo	Obtener respuestas en lenguaje natural sobre catálogo, stock y pedidos, según los permisos del usuario.
Precondiciones	Sesión iniciada; el servidor Ollama debe estar en línea con el modelo configurado.
Postcondiciones	La conversación queda persistida en la base de datos y puede retomarse luego.
Prioridad	Baja (funcionalidad opcional de nivel avanzado).

Escenario principal (resumen):

1. El usuario abre el widget "GG" y escribe una consulta (por ejemplo, "¿qué teclados tenés en stock?").
2. El sistema envía la conversación al modelo de lenguaje, que decide invocar herramientas internas (búsqueda de productos, consulta de stock, etc.) **filtradas según los permisos del usuario**.
3. La respuesta se transmite token a token en tiempo real (streaming vía SignalR) y se muestra con formato.

La arquitectura completa del chatbot (herramientas disponibles, ciclo de tool-calling, configuración del modelo) se desarrolla en [09 — Chatbot con IA](#).

Arquitectura

4. Arquitectura de la solución

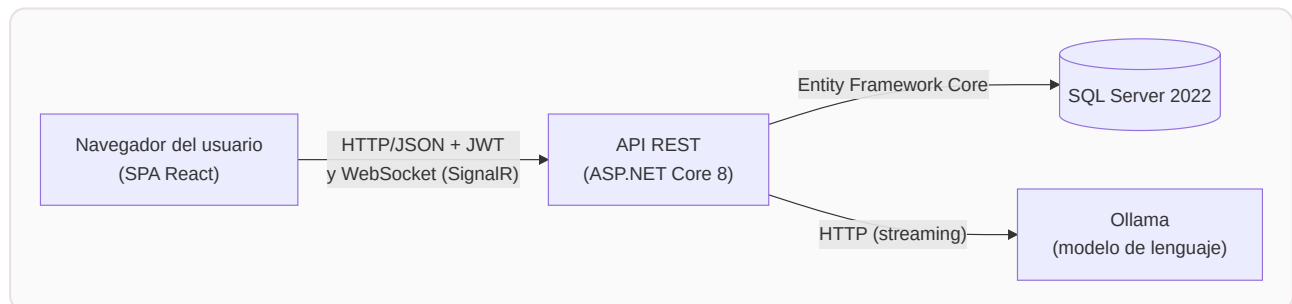
Este documento describe la arquitectura de VentaGamer en tres niveles de detalle: primero la **vista de despliegue** (los contenedores Docker y cómo se comunican), luego la **arquitectura interna del backend** (Clean Architecture) y la **del frontend** (SPA por features), y finalmente el **stack tecnológico** con la explicación de cada componente.

4.1. Visión general: arquitectura cliente-servidor

VentaGamer es una aplicación **cliente-servidor** de tres capas:

- El **cliente** es una SPA (*Single Page Application*) en React que corre en el navegador. Renderiza la interfaz, maneja la navegación y consume datos por HTTP.
- El **servidor** es una API REST en ASP.NET Core que concentra toda la lógica de negocio, la seguridad y el acceso a datos.
- La **base de datos** es SQL Server, un motor relacional donde persiste toda la información.

El cliente nunca accede a la base de datos: toda interacción pasa por la API, que valida la identidad (JWT) y los permisos en cada solicitud.



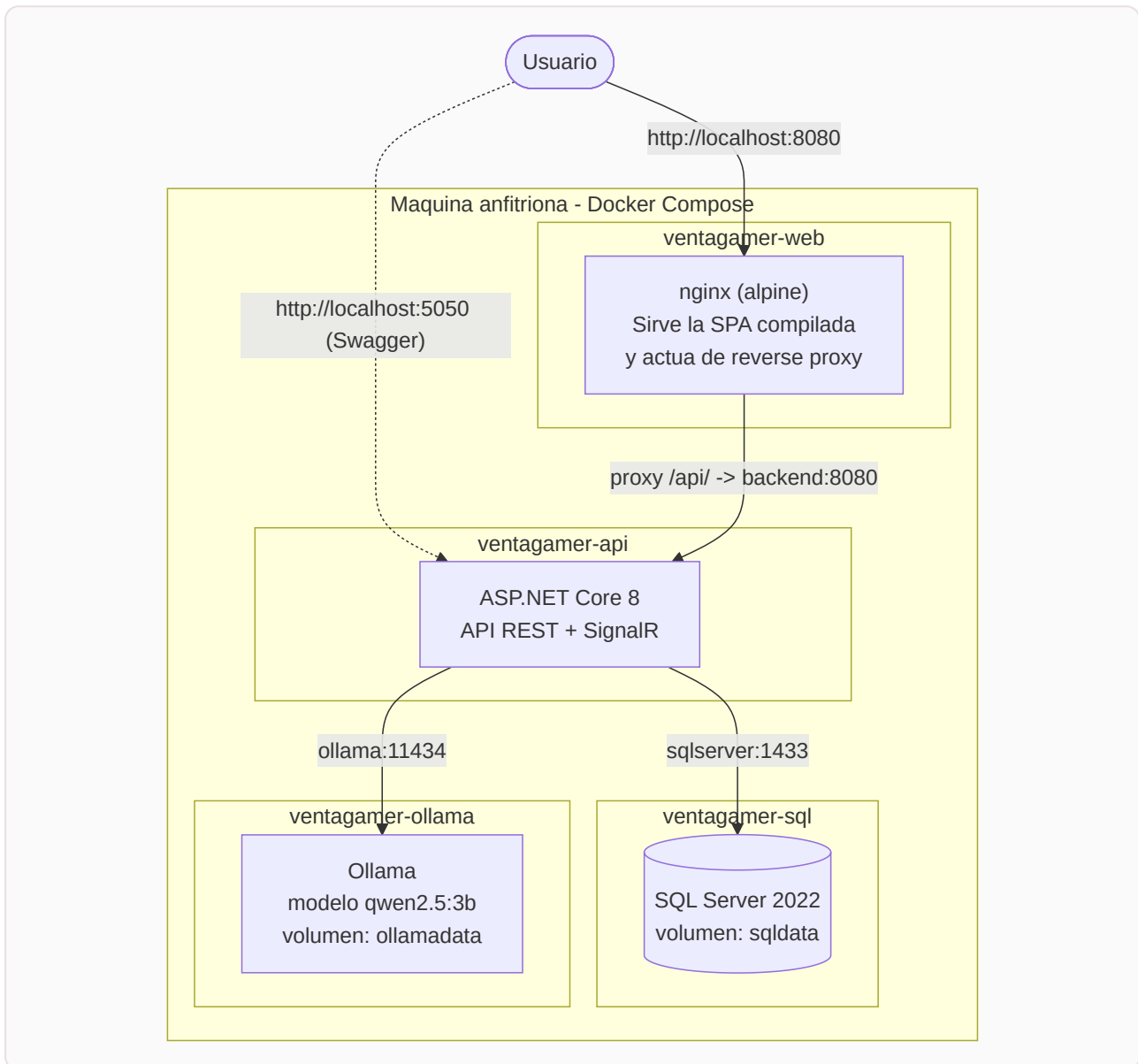
Flujo de datos de una solicitud típica

1. El usuario realiza una acción en la interfaz (por ejemplo, buscar "teclado").
2. El frontend arma la solicitud HTTP (`GET /api/products?search=teclado`) y el interceptor de Axios adjunta el token JWT en el encabezado `Authorization` .
3. La API valida el token, verifica los permisos requeridos por el endpoint (políticas de autorización) y aplica el límite de solicitudes por IP.
4. El *controller* delega en un *service* de la capa de infraestructura, que consulta la base mediante Entity Framework Core (consultas parametrizadas con paginación e índices).
5. El resultado se mapea a un DTO y vuelve como JSON.

6. TanStack Query almacena la respuesta en caché en el navegador y React actualiza el DOM solo donde cambió.

4.2. Vista de despliegue (Docker)

Todo el sistema se ejecuta como **cuatro contenedores** orquestados por Docker Compose (`docker-compose.yml`). Los contenedores se comunican entre sí por una red interna de Docker usando sus nombres de servicio como hostname; solo los puertos indicados quedan expuestos a la máquina anfitriona.



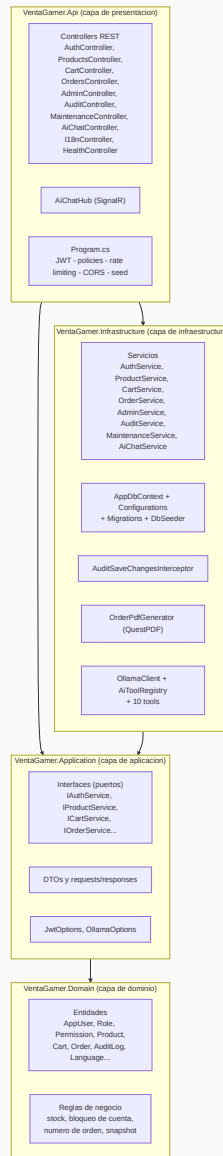
Contenedor	Imagen	Puerto host	Rol
ventagamer-web	nginx:alpine (build propio)	8080 → 80	Sirve los archivos estáticos de la SPA (HTML/CSS/JS compilados por Vite) y reenvía las llamadas <code>/api/</code> al backend.
ventagamer-api	mcr.microsoft.com/dotnet/aspnet:8.0 (build propio)	5050 → 8080	API REST, autenticación, lógica de negocio, SignalR.
ventagamer-sql	mcr.microsoft.com/mssql/server:2022-latest	1434 → 1433	Base de datos relacional. Persiste en el volumen <code>sqldata</code> .
ventagamer-ollama	ollama/ollama:latest	11434 → 11434	Motor de inferencia del modelo de lenguaje para el chatbot. Persiste los modelos en el volumen <code>ollamadata</code> .

Detalles relevantes de la orquestación:

- **Dependencias con healthcheck:** el backend no arranca hasta que SQL Server y Ollama respondan sus verificaciones de salud, evitando errores de conexión en el arranque.
- **Builds multi-etapa:** tanto el backend (`backend/Dockerfile`) como el frontend (`frontend/Dockerfile`) compilan en una imagen SDK y publican solo los artefactos finales en una imagen liviana de runtime.
- **Configuración por variables de entorno:** cadena de conexión, clave de firma JWT y modelo de IA se inyectan desde el archivo `.env`, sin tocar el código.
- **Migración y seed automáticos:** al arrancar, la API aplica las migraciones pendientes y puebla los datos iniciales (roles, permisos, usuarios demo, productos), dejando el sistema listo para usar.

4.3. Arquitectura del backend: Clean Architecture

El backend está organizado según **Clean Architecture** en cuatro proyectos con dependencias unidireccionales: las capas externas dependen de las internas y nunca al revés. El dominio no conoce la base de datos ni el framework web.



Proyecto	Responsabilidad	Depende de
VentaGamer.Domain	Entidades de negocio y sus reglas (por ejemplo: <code>Product</code> valida que el stock no sea negativo; <code>AppUser</code> implementa el bloqueo tras 3 intentos fallidos; <code>Order</code> genera su número único). No referencia ningún framework.	—
VentaGamer.Application	Contratos (interfaces) de los servicios, DTOs de entrada/salida y opciones de configuración. Define qué hace el sistema sin decir cómo .	Domain
VentaGamer.Infrastructure	Implementaciones concretas: acceso a datos con EF Core, autenticación (PBKDF2 + JWT), generación de PDF, auditoría automática, cliente de Ollama, backups.	Application, Domain
VentaGamer.Api	Capa HTTP: controllers REST, hub de SignalR, configuración de seguridad y middleware (<code>Program.cs</code>). Traduce solicitudes HTTP en llamadas a servicios.	Application, Infrastructure

Beneficios concretos de esta separación:

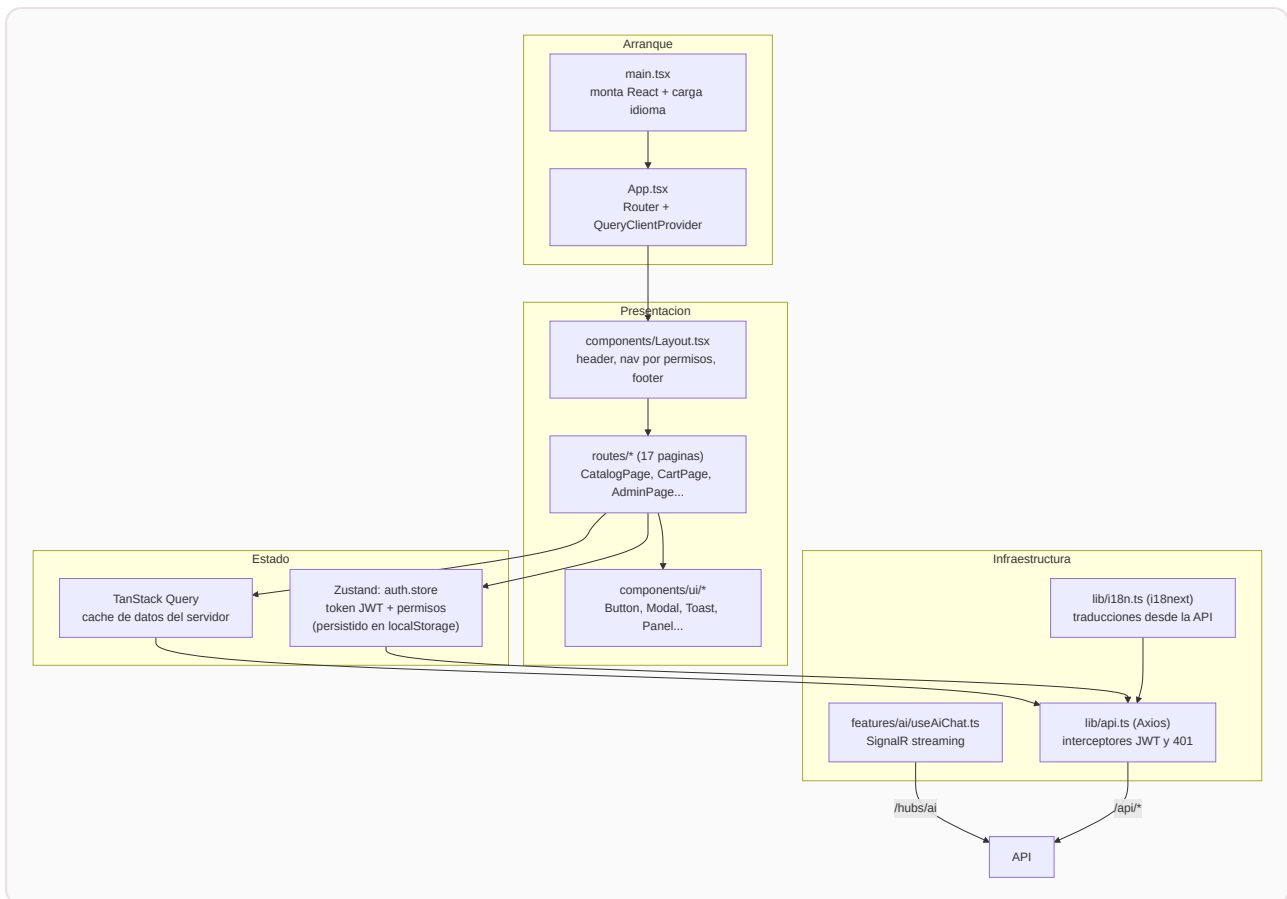
- Las reglas de negocio se pueden probar sin base de datos ni servidor web.
- Cambiar el motor de base de datos o el proveedor de IA afecta solo a Infrastructure.
- Los controllers son delgados: reciben, delegan y responden; la lógica vive en los servicios.

Patrones de diseño aplicados

Patrón	Dónde	Para qué
Inyección de dependencias	Todo el backend (<code>DependencyInjection.cs</code>)	Los controllers reciben interfaces; las implementaciones se registran en un único lugar.
Repository implícito / Unit of Work	<code>AppDbContext</code> de EF Core	El DbContext agrupa los cambios y los confirma como unidad.
Interceptor	<code>AuditSaveChangesInterceptor</code>	Audita automáticamente todo cambio en entidades sensibles sin código repetido en cada servicio.
Snapshot	<code>OrderItem</code>	Congela título y precio del producto al momento de la compra.
Composite	<code>RoleHierarchy</code>	Un rol hereda los permisos de sus roles padre (recorrido BFS al calcular permisos efectivos).
Strategy / Options	<code>JwtOptions</code> , <code>OllamaOptions</code>	Configuración tipada e intercambiable sin recompilar.
Tool calling (function calling)	<code>AiToolRegistry</code> + tools	El modelo de lenguaje invoca funciones del sistema de forma controlada y filtrada por permisos.

4.4. Arquitectura del frontend: SPA por features

El frontend es una SPA en React 19 + TypeScript organizada por **funcionalidades verticales** (*feature folders*): cada dominio (auth, productos, carrito, admin, IA) agrupa su API, sus tipos y sus componentes.



Decisiones principales:

- **Estado de servidor vs. estado de sesión separados:** los datos que vienen de la API (productos, carrito, pedidos) viven en la caché de TanStack Query, que maneja recarga, invalidación y estados de carga/error. La sesión (token + usuario + permisos) vive en un único store de Zustand persistido en `localStorage`.
- **Autorización en la interfaz por permisos:** el menú y las páginas consultan `hasPermission(código)` para mostrar u ocultar opciones. Esta verificación es solo de **experiencia de usuario**: la garantía real está en el servidor, que revalida permisos en cada endpoint.
- **Ciente HTTP centralizado:** una única instancia de Axios adjunta el JWT en cada solicitud y, ante una respuesta 401 (token vencido), cierra la sesión automáticamente.
- **Tiempo real:** el chat con el asistente IA usa SignalR (WebSocket) para recibir la respuesta token a token, en lugar de esperar la respuesta completa.

4.5. Stack tecnológico

Tabla de referencia de todas las tecnologías utilizadas, su rol en el sistema y una breve descripción para quien no las conozca.

Backend

Tecnología	Versión	Rol en VentaGamer	Qué es
.NET 8 (LTS)	8.0	Plataforma de ejecución del backend.	Plataforma de desarrollo de Microsoft, multiplataforma (Windows/Linux/macOS) y de código abierto. La versión 8 es de soporte extendido.
ASP.NET Core	8.0	Framework de la API REST: controllers, middleware, autenticación, autorización.	Framework web de .NET para construir APIs y aplicaciones web de alto rendimiento.
Entity Framework Core	8.0.10	Acceso a datos: mapea las entidades C# a tablas SQL, genera las migraciones y traduce consultas LINQ a SQL.	ORM (mapeador objeto-relacional) oficial de .NET. Permite trabajar con la base de datos usando objetos en lugar de SQL manual.
SQL Server 2022	Developer	Base de datos relacional del sistema.	Motor de base de datos relacional de Microsoft; aquí corre en Linux dentro de Docker.
SignalR	8.0.10	Comunicación en tiempo real para el streaming del chatbot.	Librería de ASP.NET Core que abstrae WebSockets para enviar mensajes servidor → cliente al instante.
JWT (JSON Web Tokens)	—	Credencial de sesión: el login emite un token firmado que el cliente presenta en cada solicitud.	Estándar abierto (RFC 7519) de tokens autofirmados que contienen los datos y permisos del usuario.
QuestPDF	2024.10.1	Generación de los comprobantes de compra en PDF.	Librería .NET de código abierto para componer documentos PDF mediante código C#.
Swagger / Swashbuckle	6.6.2	Documentación interactiva de la API (entorno de desarrollo).	Herramienta que genera una interfaz web para explorar y probar los endpoints.

Frontend

Tecnología	Versión	Rol en VentaGamer	Qué es
React	19.2	Construcción de la interfaz: componentes, estado y actualización eficiente del DOM.	Librería de JavaScript para construir interfaces mediante componentes declarativos.
TypeScript	6.0	Lenguaje del frontend: agrega tipos estáticos a JavaScript.	Superconjunto de JavaScript de Microsoft que detecta errores de tipos en tiempo de compilación.
Vite	8.0	Servidor de desarrollo (con recarga instantánea) y empaquetador de producción.	Herramienta de build moderna, sucesora de Webpack en la mayoría de los proyectos nuevos.
Tailwind CSS	3.4	Estilos y diseño responsivo mediante clases utilitarias.	Framework CSS que compone los estilos con clases pequeñas (<code>p-4</code> , <code>md:flex</code>) en lugar de hojas de estilo separadas.
React Router	7.15	Navegación entre páginas de la SPA sin recargar el navegador.	Librería estándar de enrutamiento del ecosistema React.
TanStack Query	5.100	Caché y sincronización de los datos que provienen de la API.	Librería de manejo de "estado de servidor": deduplica solicitudes, cachea respuestas y maneja estados de carga y error.
Zustand	5.0	Almacenamiento de la sesión (token JWT, usuario, permisos).	Librería minimalista de estado global para React.
Axios	1.16	Cliente HTTP con interceptores (adjuntar JWT, capturar 401).	Librería popular para realizar solicitudes HTTP desde el navegador.
i18next / react-i18next	26 / 17	Multi-idioma (es/en/pt) con traducciones cargadas desde la API.	Framework de internacionalización estándar del ecosistema JavaScript.
@microsoft/signalr	10.0	Cliente WebSocket para el streaming del chatbot.	Cliente JavaScript oficial de SignalR.

Infraestructura e IA

Tecnología	Rol en VentaGamer	Qué es
Docker / Docker Compose	Empaqueta y orquesta los 4 servicios del sistema (web, API, base de datos, IA) con un solo comando.	Plataforma de contenedores: cada servicio corre aislado con sus dependencias, garantizando que funcione igual en cualquier máquina.
nginx	Sirve los archivos estáticos del frontend y reenvía las llamadas <code>/api/</code> al backend (reverse proxy).	Servidor web liviano y de alto rendimiento, estándar de facto para servir SPAs.
Ollama	Ejecuta localmente el modelo de lenguaje del chatbot (sin depender de servicios en la nube).	Herramienta de código abierto para correr modelos de lenguaje (LLM) en hardware propio, con una API HTTP compatible con tool calling.
qwen2.5:3b	Modelo de lenguaje del asistente GG.	Modelo compacto (~2 GB) de la familia Qwen 2.5, elegido por su bajo consumo de recursos y su soporte de invocación de herramientas (function calling).

Base de Datos

5. Base de datos

VentaGamer utiliza **SQL Server 2022** como base de datos relacional. Este documento presenta el modelo de datos completo: el diagrama de entidad-relación (DER), el diagrama de clases de las entidades del dominio, el detalle de cada tabla y la explicación de cómo se crea y puebla la base en la práctica.

5.1. Cómo se genera la base de datos realmente

Un punto importante para entender el proyecto: **la base de datos no se crea con un script SQL manual**, sino mediante **migraciones de Entity Framework Core** (enfoque *code-first*):

1. Las entidades se definen como clases C# en `backend/src/VentaGamer.Domain/Entities/`.
2. Las reglas de mapeo (nombres de tabla, longitudes, índices, claves foráneas, comportamiento al borrar) se definen en `backend/src/VentaGamer.Infrastructure/Persistence/Configurations/`.
3. A partir de ambas, EF Core genera **migraciones**: clases C# que describen los cambios incrementales del esquema (`backend/src/VentaGamer.Infrastructure/Persistence/Migrations/`).
4. Al arrancar, la API ejecuta `MigrateAsync()` (en `DbSeeder.SeedAsync`), que aplica las migraciones pendientes y deja el esquema actualizado. A continuación el *seeder* puebla los datos iniciales: idiomas, permisos, roles, usuarios demo, traducciones y 20 productos de ejemplo.

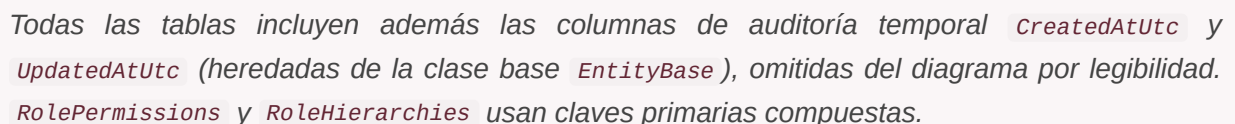
Las migraciones existentes son cuatro:

Migración	Qué crea
<code>InitialCreate</code>	Languages, Permissions, Products, Roles, RoleHierarchies, RolePermissions, Users, Translations, AuditLogs
<code>AddCartsAndOrders</code>	Carts, CartItems, Orders, OrderItems
<code>AddAiChatTables</code>	AiConversations, AiMessages
<code>AddSystemSettings</code>	SystemSettings

Sobre el script SQL de esta carpeta: el archivo `sql/ventagamer-demo.sql` es una **versión simplificada y demostrativa** del esquema, pensada para que el lector entienda con SQL estándar cómo son las tablas núcleo y sus relaciones. No es el mecanismo real de creación de la base (que son las migraciones descritas arriba) y omite tablas secundarias y detalles menores en favor de la claridad.

- El esquema queda **versionado junto al código**: cada cambio de modelo genera una migración revisable en git.
- La base se crea y actualiza **automáticamente y de forma idéntica** en cualquier entorno (desarrollo, Docker, producción).
- Se elimina la deriva entre "lo que dice el script" y "lo que espera el código".

El modelo consta de **15 tablas** organizadas en cinco áreas funcionales: seguridad (usuarios, roles, permisos), catálogo y ventas (productos, carritos, pedidos), internacionalización (idiomas, traducciones), auditoría/configuración, y chatbot IA.



Vista del modelo desde el dominio (clases C#), con sus principales métodos de negocio. La mayoría de las entidades heredan de `EntityBase` (`Id`, `CreatedAtUtc`, `UpdatedAtUtc`); se omite del diagrama para no saturarlo con flechas de herencia. `RolePermission` y `RoleHierarchy` son tablas de unión y no heredan de esa clase base.



Users — cuentas del sistema.

Columna	Tipo	Restricción
Id	int IDENTITY	PK
Username	nvarchar(60)	UNIQUE, NOT NULL
PasswordHash	nvarchar(500)	NOT NULL — hash PBKDF2, nunca texto plano
IsBlocked	bit	bloqueo manual o por intentos fallidos
FailedLoginAttempts	int	se reinicia al ingresar correctamente
LastLoginUtc	datetime2	nullable
LanguageId	int	FK → Languages (DELETE RESTRICT)
RoleId	int	FK → Roles (DELETE RESTRICT)

Índices: único sobre `Username` ; índices sobre `LanguageId` y `RoleId` .

`Roles` — perfiles de acceso (`Name` único). `Permissions` — catálogo de permisos atómicos (`Code` único, 12 filas sembradas).

`RolePermissions` — relación N:M rol ↔ permiso. PK compuesta (`RoleId` , `PermissionId`), borrado en cascada en ambos sentidos.

`RoleHierarchies` — herencia entre roles (patrón Composite). PK compuesta (`ParentRoleId` , `ChildRoleId`), DELETE RESTRICT para evitar borrados accidentales de roles con herencia.

Área de catálogo y ventas

`Products` — catálogo.

Columna	Tipo	Nota
Title	nvarchar(200)	indexado (búsqueda)
Category	nvarchar(80)	indexado (filtro)
Price	decimal(18,2)	
Stock	int	validado ≥ 0 por la entidad
ImageUrl	nvarchar(500)	nullable
IsActive	bit	baja lógica: los productos no se borran, se desactivan

`Carts` / `CartItems` — carrito persistente. `Carts.UserId` tiene índice **único**: un usuario tiene exactamente un carrito. `CartItems` tiene índice único compuesto (`CartId` , `ProductId`): un producto aparece una sola vez por carrito (se modifica la cantidad). FK a Products con DELETE RESTRICT: no puede eliminarse físicamente un producto referenciado.

Orders / OrderItems — pedidos confirmados. **OrderNumber** nvarchar(40) único (formato **VG-{fecha}-{sufijo}**); índices sobre **PlacedAtUtc** y **UserId** para consultas por fecha y por cliente. **OrderItems** guarda **ProductTitle** y **UnitPrice** **copiados al momento de la compra** (snapshot): el historial no cambia si luego se modifica el catálogo.

Área de internacionalización

Languages — idiomas disponibles (**Code** único: **es**, **en**, **pt**). **Translations** — pares clave/valor por idioma, con índice único compuesto (**LanguageId**, **TextKey**). El frontend descarga el diccionario del idioma activo desde la API.

Área de auditoría y configuración

AuditLogs — bitácora. Indexada por **EventUtc**, **Module** y **UserId** para los filtros de consulta. **UserId** es nullable con DELETE SET NULL: si se elimina un usuario, sus eventos de auditoría se conservan (sin referencia).

SystemSettings — configuración clave/valor modificable en tiempo de ejecución (**Key** único). Se usa, por ejemplo, para la URL y el modelo del servidor de IA (**Ai:BaseUrl**, **Ai:Model**), lo que permite cambiarlos desde la pantalla de administración sin reiniciar el sistema.

Área de chatbot

AiConversations — conversaciones por usuario (borrado en cascada). **AiMessages** — mensajes de cada conversación con su rol (usuario / asistente / herramienta / sistema), contenido y, si corresponde, el JSON de las herramientas invocadas; índice compuesto (**ConversationId**, **CreatedAtUtc**) para recuperar el historial en orden.

5.5. Consultas eficientes y volúmenes de datos

El diseño contempla el crecimiento del volumen de datos con tres mecanismos:

1. **Índices dirigidos a las consultas reales.** Cada filtro de la aplicación tiene su índice de respaldo: búsqueda de productos (**Title**, **Category**, **IsActive**), pedidos por cliente y por fecha (**UserId**, **PlacedAtUtc**), bitácora por fecha/módulo/usuario, y unicidad garantizada por índices únicos (**Username**, **OrderNumber**, carrito por usuario).
2. **Paginación en el servidor.** Los listados (catálogo, pedidos, bitácora) nunca devuelven tablas completas: la API recibe **page** y **pageSize** y resuelve con **Skip/Take**, devolviendo también el total para que el cliente arme la paginación. Así, el costo de cada consulta se mantiene acotado aunque la tabla crezca.
3. **Consultas de solo lectura optimizadas.** Las consultas que no modifican datos usan **AsNoTracking()** de EF Core, evitando el costo de seguimiento de cambios.

La **simulación de volúmenes** se realiza mediante el seeder (20 productos en 9 categorías, 5 usuarios con roles distintos, traducciones en 3 idiomas) y crece orgánicamente con el uso: cada compra genera

pedidos e ítems, cada operación administrativa genera filas de bitácora, y cada conversación con el chatbot persiste sus mensajes.

5.6. Integridad referencial: resumen de políticas de borrado

Relación	Política	Justificación
Users → Roles / Languages	RESTRICT	No puede eliminarse un rol o idioma con usuarios asignados.
CartItems / OrderItems → Products	RESTRICT	Protege el historial: un producto referenciado no se borra físicamente (se usa baja lógica).
Carts → Users, CartItems → Carts	CASCADE	El carrito y sus ítems no tienen valor sin el usuario.
OrderItems → Orders	CASCADE	Los ítems pertenecen al pedido.
Orders → Users	RESTRICT	Los pedidos son registros contables: no deben desaparecer en cascada.
AuditLogs → Users	SET NULL	La bitácora se conserva aunque el usuario se elimine.
AiConversations → Users, AiMessages → AiConversations	CASCADE	Las conversaciones son datos personales del usuario.

APIs

6. APIs

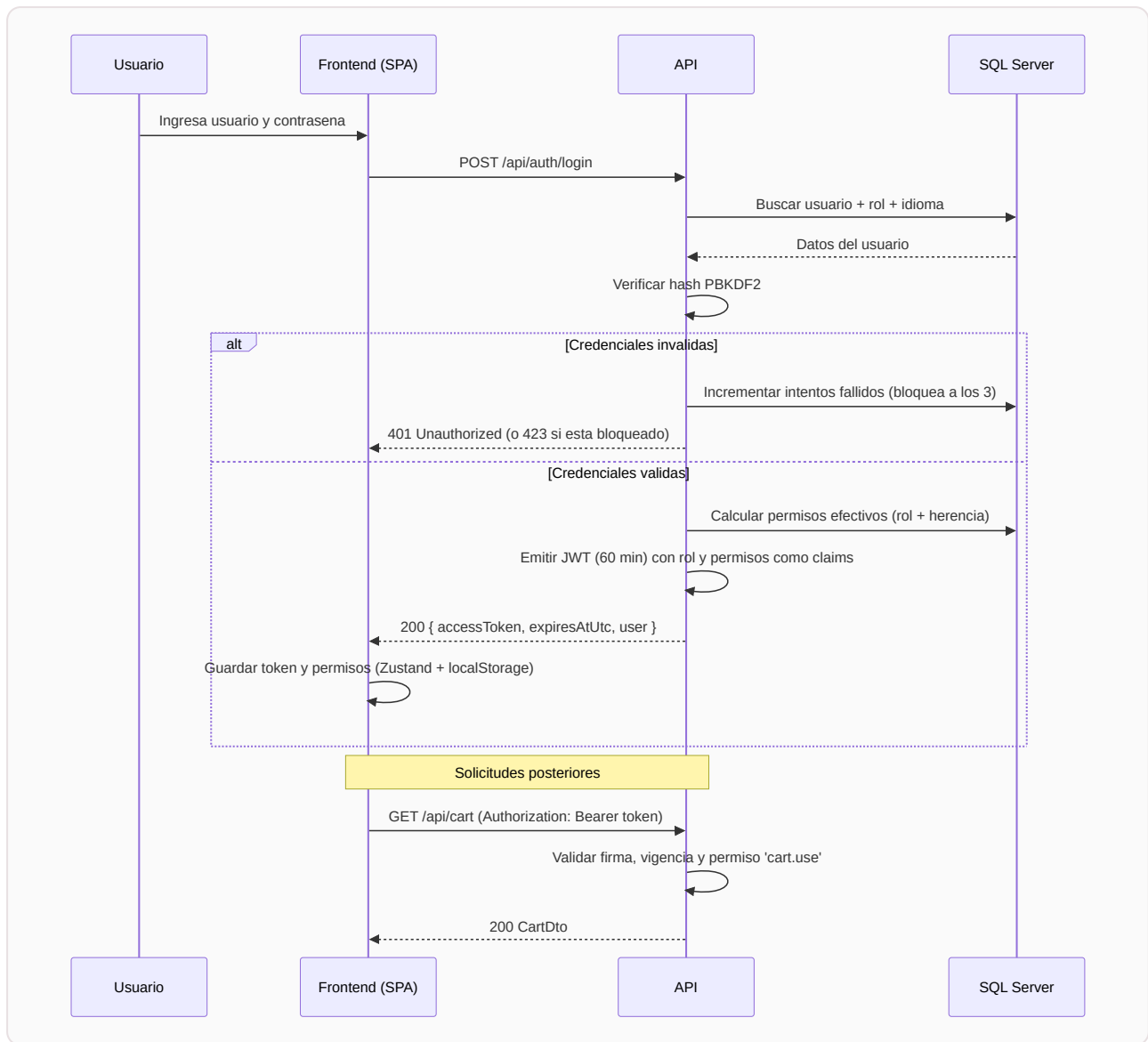
Toda la funcionalidad de VentaGamer se expone a través de una **API REST** propia, desarrollada en ASP.NET Core, que el frontend consume mediante HTTP/JSON. A esto se suman dos integraciones adicionales: un canal **WebSocket (SignalR)** para el streaming del chatbot y el consumo de la **API HTTP de Ollama** desde el backend para la inferencia del modelo de lenguaje.

6.1. Convenciones generales

- **Base:** todas las rutas cuelgan de `/api`. En desarrollo el backend escucha en `http://localhost:5050`; en Docker, nginx reenvía `/api/` al contenedor del backend.
- **Formato:** solicitudes y respuestas en JSON (`Content-Type: application/json`).
- **Autenticación:** los endpoints protegidos exigen el encabezado `Authorization: Bearer {token JWT}`.
- **Autorización:** además del token, muchos endpoints exigen un **permiso** puntual (columna "Autorización" en las tablas siguientes). El servidor valida el permiso en cada solicitud mediante políticas; no confía en lo que la interfaz muestre u oculte.
- **Exploración:** en entorno de desarrollo, Swagger UI está disponible en `http://localhost:5050/swagger` para probar los endpoints de forma interactiva.

6.2. Autenticación: flujo JWT

El sistema usa **JSON Web Tokens**: al iniciar sesión, el servidor emite un token firmado (HMAC-SHA256) que contiene la identidad del usuario, su rol y **todos sus permisos efectivos** como claims. El cliente lo guarda y lo presenta en cada solicitud; el servidor lo valida sin necesidad de consultar la base en cada acceso.



Detalles de seguridad del flujo:

- Contraseñas almacenadas con **PBKDF2** (hasher de ASP.NET Identity, con salt individual).
- **Bloqueo automático** de la cuenta al tercer intento fallido consecutivo (HTTP 423).
- Token con vigencia de **60 minutos**; al expirar, el interceptor del frontend detecta el 401 y cierra la sesión.
- Los endpoints de login y registro tienen un límite de **5 solicitudes por minuto por IP** contra fuerza bruta.

6.3. Referencia de endpoints

Autenticación — `/api/auth` (`AuthController`)

Método	Ruta	Autorización	Descripción
POST	<code>/api/auth/login</code>	Anónimo	Inicia sesión. Devuelve token JWT, expiración y datos del usuario. Errores: 401 (credenciales), 423 (cuenta bloqueada), 429 (límite de intentos).
POST	<code>/api/auth/register</code>	Anónimo	Registra un usuario nuevo (rol Cliente). Error 409 si el nombre ya existe.
GET	<code>/api/auth/me</code>	Autenticado	Devuelve el perfil del usuario del token.

Catálogo — `/api/products` (`ProductsController`)

Método	Ruta	Autorización	Descripción
GET	<code>/api/products</code>	Anónimo	Listado paginado. Parámetros: <code>page</code> , <code>pageSize</code> , <code>search</code> (texto), <code>category</code> . Devuelve ítems + total.
GET	<code>/api/products/{id}</code>	Anónimo	Detalle de un producto (404 si no existe).
GET	<code>/api/products/categories</code>	Anónimo	Lista de categorías activas.
POST	<code>/api/products</code>	<code>products.write</code>	Alta de producto.
PUT	<code>/api/products/{id}</code>	<code>products.write</code>	Modificación.
DELETE	<code>/api/products/{id}</code>	<code>products.write</code>	Baja lógica (desactiva; no borra).

Carrito — `/api/cart` (`cartController` , todo requiere `cart.use`)

Método	Ruta	Descripción
GET	<code>/api/cart</code>	Carrito del usuario (se crea al primer uso).
POST	<code>/api/cart/items</code>	Agrega un producto (<code>productId</code> , <code>quantity</code>); valida stock.
PUT	<code>/api/cart/items/{id}</code>	Cambia la cantidad de un ítem.
DELETE	<code>/api/cart/items/{id}</code>	Quita un ítem.
DELETE	<code>/api/cart</code>	Vacía el carrito.

Pedidos — `/api/orders` (`ordersController` , requiere autenticación)

Método	Ruta	Autorización	Descripción
POST	<code>/api/orders/checkout</code>	<code>cart.use</code>	Transaccional: valida stock, lo descuenta, crea el pedido y vacía el carrito. Ante cualquier falla revierte todo.
GET	<code>/api/orders/mine</code>	Autenticado	Pedidos del usuario actual.
GET	<code>/api/orders</code>	<code>orders.read.all</code>	Todos los pedidos (filtro opcional <code>username</code>).
GET	<code>/api/orders/{id}</code>	Dueño o <code>orders.read.all</code>	Detalle de un pedido.
GET	<code>/api/orders/{id}/pdf</code>	Dueño o <code>orders.read.all</code>	Comprobante PDF generado con QuestPDF.

Administración — `/api/admin` (`AdminController`)

Método	Ruta	Autorización	Descripción
GET	<code>/api/admin/roles</code>	<code>roles.read</code>	Roles con permisos y roles padre.
GET	<code>/api/admin/permissions</code>	<code>roles.read</code>	Catálogo de los 12 permisos.
POST	<code>/api/admin/roles</code>	<code>roles.write</code>	Crear rol (permisos + herencia).
PUT	<code>/api/admin/roles/{id}</code>	<code>roles.write</code>	Modificar rol.
DELETE	<code>/api/admin/roles/{id}</code>	<code>roles.write</code>	Eliminar rol (409 si tiene usuarios).
GET	<code>/api/admin/users</code>	<code>users.register</code>	Listar usuarios.
PUT	<code>/api/admin/users/{id}/blocked</code>	<code>users.register</code>	Bloquear / desbloquear.
PUT	<code>/api/admin/users/{id}/role</code>	<code>users.register</code>	Cambiar rol de un usuario.

Bitácora — `/api/audit` (`AuditController`)

Método	Ruta	Autorización	Descripción
GET	<code>/api/audit</code>	<code>audit.read</code>	Bitácora paginada; filtros: <code>username</code> , <code>module</code> , <code>from</code> , <code>to</code> .

Mantenimiento — `/api/maintenance` (`MaintenanceController`)

Método	Ruta	Autorización	Descripción
POST	<code>/api/maintenance/backup</code>	<code>backup.manage</code>	Genera un backup <code>.bak</code> comprimido de la base.
GET	<code>/api/maintenance/backups</code>	<code>backup.manage</code>	Historial de backups (desde el catálogo <code>msdb</code>).
GET	<code>/api/maintenance/integrity</code>	<code>integrity.check</code>	Verificación de integridad HMAC-SHA256 de usuarios y pedidos.

Chatbot IA — `/api/ai` (`AIChatController` , requiere autenticación)

Método	Ruta	Autorización	Descripción
GET	<code>/api/ai/status</code>	Autenticado	¿Está Ollama disponible?
GET	<code>/api/ai/conversations</code>	Autenticado	Conversaciones del usuario.
POST	<code>/api/ai/conversations</code>	Autenticado	Nueva conversación.
GET	<code>/api/ai/conversations/{id}/messages</code>	Autenticado (dueño)	Historial de mensajes.
DELETE	<code>/api/ai/conversations/{id}</code>	Autenticado (dueño)	Eliminar conversación.
GET	<code>/api/ai/config</code>	<code>config.read</code>	Configuración del asistente (URL, modelo, modelos disponibles).
PUT	<code>/api/ai/config</code>	<code>roles.write</code>	Cambiar URL/modelo en tiempo de ejecución (persiste en <code>SystemSettings</code>).
POST	<code>/api/ai/config/test</code>	<code>config.read</code>	Probar conexión contra una URL sin guardarla.

Internacionalización — `/api/i18n` (`I18nController`)

Método	Ruta	Autorización	Descripción
GET	<code>/api/i18n/languages</code>	Anónimo	Idiomas disponibles.
GET	<code>/api/i18n/translations/{lang}</code>	Anónimo	Diccionario clave → valor del idioma.

Salud — `/api/health` (`HealthController`)

Método	Ruta	Autorización	Descripción
GET	<code>/api/health</code>	Anónimo	Estado del servicio y conectividad con la base de datos.

6.4. Canal en tiempo real: SignalR

Para el chatbot, la respuesta del modelo se transmite **token a token** por un hub de SignalR (WebSocket), lo que permite mostrar el texto a medida que se genera en lugar de esperar la respuesta completa:

- **Hub:** `/hubs/ai` (`AiChatHub`). Autenticación por JWT enviado como parámetro de conexión (`access_token`), como exige WebSocket.
- **Cliente → servidor:** `SendMessage(conversationId, text)`.
- **Servidor → cliente:** eventos `AiStreamStart` (inicio), `AiStreamToken` (cada fragmento de texto), `AiStreamEnd` (fin) y `AiStreamError` (por ejemplo, si Ollama está fuera de línea).
- **Protección propia:** máximo 10 mensajes por minuto por conexión.

6.5. Manejo de errores

La API comunica los errores mediante códigos HTTP estándar y mensajes JSON; el frontend los normaliza con un helper (`toApiError`) y los muestra de forma amigable.

Código	Cuándo ocurre	Cómo lo maneja el frontend
400	Datos inválidos (por ejemplo, URL de Ollama mal formada).	Mensaje de validación junto al formulario.
401	Sin token, token vencido o credenciales incorrectas.	El interceptor de Axios cierra la sesión automáticamente y redirige al login.
403	Token válido pero sin el permiso requerido.	Pantalla de "sin permisos" (estado vacío).
404	Recurso inexistente (producto, pedido, conversación).	Mensaje de "no encontrado".
409	Conflicto: usuario ya existente, rol con usuarios asignados.	Mensaje específico del conflicto.
423	Cuenta bloqueada por intentos fallidos.	Mensaje indicando contactar al administrador.
429	Límite de solicitudes excedido (rate limiting).	Mensaje de "demasiados intentos, aguarde".
500	Error interno (por ejemplo, falla de la base).	Mensaje genérico con opción de reintentar.

Rate limiting (protección contra abuso): configurado en `Program.cs` con el middleware nativo de ASP.NET Core:

- **Global:** 60 solicitudes por minuto por dirección IP.
- **Endpoints de autenticación:** 5 solicitudes por minuto por IP (login y registro).
- Al excederse, el servidor responde **429 Too Many Requests**.

Resiliencia hacia la base de datos: la conexión de EF Core usa `EnableRetryOnFailure` (hasta 3 reintentos automáticos ante fallas transitorias), y el checkout usa la estrategia de ejecución de EF combinada con transacciones explícitas.

6.6. Cómo consume la API el frontend

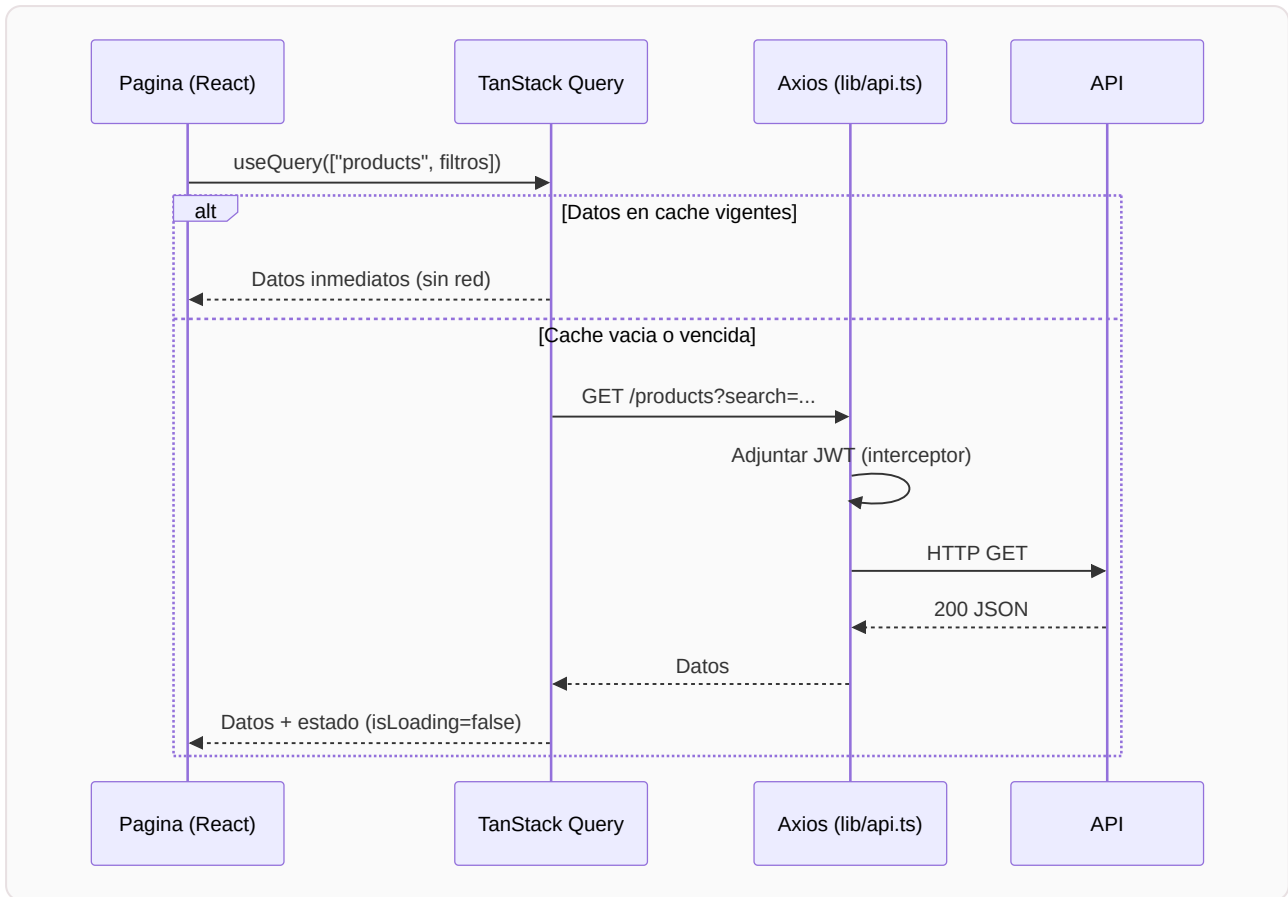
El frontend nunca usa `fetch` disperso por el código: centraliza el acceso en dos piezas.

1. Cliente HTTP único (Axios) — `frontend/src/lib/api.ts`:

- Instancia con `baseUrl: "/api"` (en desarrollo, Vite hace proxy a `localhost:5050`; en Docker, nginx hace proxy al backend).
- Interceptor de solicitud: adjunta `Authorization: Bearer {token}` leyendo el store de sesión.
- Interceptor de respuesta: ante un 401 con sesión activa, ejecuta el logout automático.

2. TanStack Query — en cada página:

- Las **consultas** (`useQuery`) cachean los datos con una clave (por ejemplo `["products", page, search]`), evitan solicitudes duplicadas y exponen estados de carga y error listos para la interfaz.
- Las **mutaciones** (`useMutation`) ejecutan las escrituras (agregar al carrito, checkout, ABM) e **invalidan** las consultas afectadas para que la interfaz se actualice sola.



6.7. API externa consumida: Ollama

Además de exponer su propia API, el backend **consume una API externa**: la de **Ollama**, el servidor local de modelos de lenguaje.

- `GET {ollama}/api/tags` — verificación de disponibilidad y listado de modelos instalados.
- `POST {ollama}/api/chat` — inferencia en modo streaming, con soporte de *tool calling* (el modelo puede pedir la ejecución de funciones del sistema).

La URL y el modelo se leen en cada solicitud desde la tabla `SystemSettings` (con valores por defecto en la configuración), por lo que pueden cambiarse desde la pantalla de administración **sin reiniciar** el sistema. El detalle completo está en [09 — Chatbot con IA](#).

Frontend

7. Frontend: interfaz, DOM, responsive y formularios

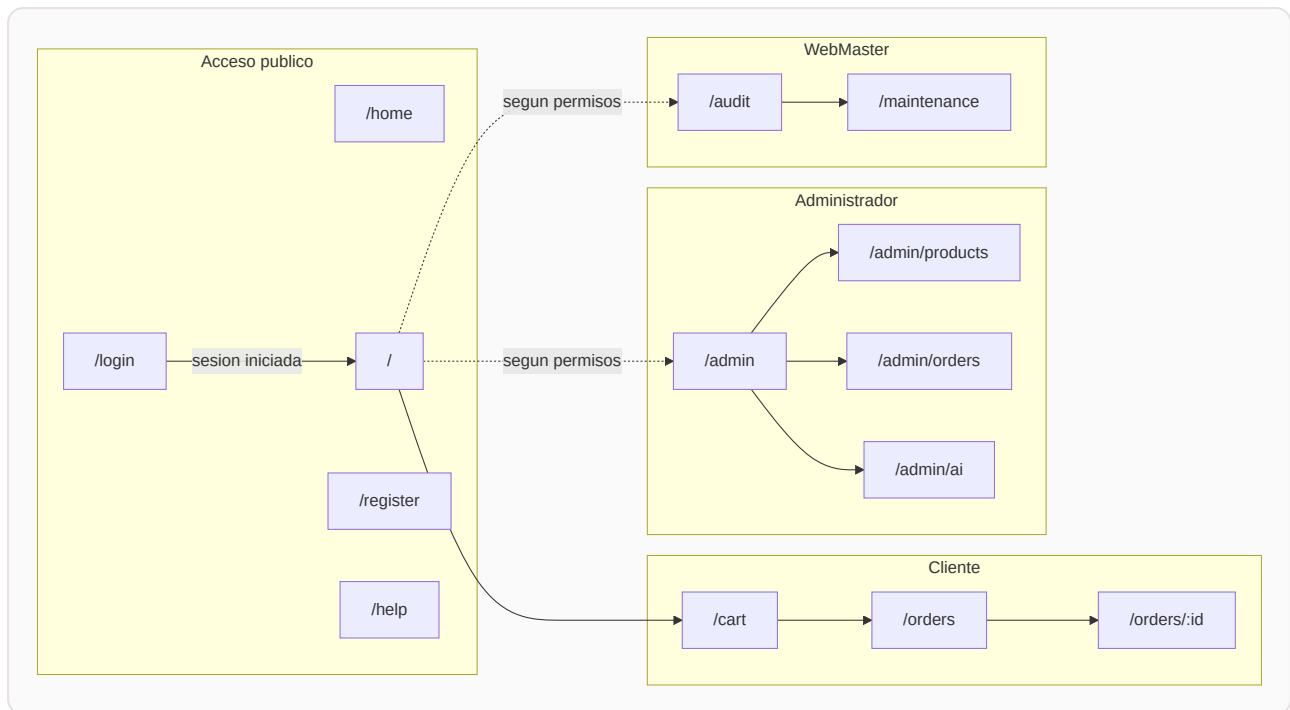
El frontend de VentaGamer es una **SPA (Single Page Application)** construida con React 19, TypeScript, Vite y Tailwind CSS. Este documento describe la estructura de páginas y navegación, cómo se manipula el DOM, el diseño responsive, los formularios con sus validaciones, el sistema visual y el soporte multi-idioma.

7.1. Páginas y navegación

La aplicación tiene **17 páginas** gestionadas por React Router. La navegación es **interna**: al cambiar de página no se recarga el navegador; el router intercambia los componentes dentro del layout común (encabezado con menú, contenido, pie).

Ruta	Página	Acceso
/	Catálogo de productos (página principal)	Público
/home	Landing de presentación	Público
/login	Inicio de sesión	Público
/register	Registro de cuenta	Público
/password-reset	Recuperación de contraseña (demostrativa)	Público
/help	Ayuda y preguntas frecuentes	Público
/cart	Carrito y checkout	Sesión + <code>cart.use</code>
/orders	Mis compras	Sesión
/orders/:id	Detalle de pedido + descarga PDF	Sesión (dueño o admin)
/admin	Gestión de roles y usuarios	<code>roles.read</code> o <code>users.register</code>
/admin/products	ABM de productos	<code>products.write</code>
/admin/orders	Todas las compras	<code>orders.read.all</code>
/admin/ai	Configuración del asistente IA	<code>config.read</code> (edición: <code>roles.write</code>)
/audit	Bitácora de auditoría	<code>audit.read</code>
/maintenance	Backups e integridad	<code>backup.manage</code> o <code>integrity.check</code>
/config	Preferencias del usuario	Sesión
*	Página 404	Público

El **menú se construye dinámicamente según los permisos** del usuario: un cliente ve Catálogo, Carrito y Mis compras; un administrador ve además Administración; un WebMaster ve Bitácora y Mantenimiento. La verificación en la interfaz es de usabilidad — la garantía real la da el servidor, que revalida permisos en cada endpoint (ver **06 — APIs**).



7.2. Manipulación del DOM

La consigna exige manipulación del DOM (crear, modificar y eliminar elementos, y responder a eventos). En VentaGamer esto se realiza a través de **React**, que es hoy la forma profesional estándar de manipular el DOM: en lugar de llamar manualmente a `document.createElement` o `element.remove()`, se **declara** cómo debe verse la interfaz para cada estado, y React calcula y aplica las operaciones de DOM mínimas necesarias (mediante su DOM virtual).

Ejemplos concretos en la aplicación:

Interacción	Qué ocurre en el DOM	Dónde
Escribir en el buscador del catálogo	Se eliminan las tarjetas de producto que no coinciden y se crean las nuevas, sin recargar la página.	<code>CatalogPage.tsx</code>
Agregar un producto al carrito	El contador del carrito en el encabezado se modifica al instante; aparece un aviso (toast) que luego se elimina solo.	<code>Layout.tsx</code> , <code>Toast</code>
Cambiar cantidades en el carrito	Cada botón +/- dispara un evento que actualiza los nodos del total y de la línea.	<code>CartPage.tsx</code>
Abrir el detalle de una imagen	Se crea un visor superpuesto (lightbox) y se destruye al presionar Escape o hacer clic afuera.	<code>Lightbox</code>
Abrir el formulario de producto	Se monta un modal con el formulario; al guardar o cancelar se desmonta .	<code>AdminProductsPage.tsx</code> , <code>Modal</code>
Conversar con el asistente IA	Cada token recibido por WebSocket agrega texto al último mensaje en pantalla, en tiempo real.	<code>AiChatWidget.tsx</code>
Abrir el menú en un móvil	El botón hamburguesa crea/oculta el panel de navegación.	<code>Layout.tsx</code>

El **manejo de eventos** (clic, envío de formularios, teclado, cambio de inputs) se realiza con los manejadores sintéticos de React (`onClick` , `onSubmit` , `onChange` , `onKeyDown`), que por debajo registran los listeners nativos del navegador.

7.3. Diseño responsivo

El diseño se adapta a móviles, tablets y escritorio usando las **utilidades responsivas de Tailwind CSS**, que aplican estilos según puntos de corte (*breakpoints*): `sm` (≥640 px), `md` (≥768 px), `lg` (≥1024 px), etc. El enfoque es *mobile-first*: los estilos base apuntan a pantallas chicas y los prefijos agregan variantes para pantallas mayores.

Aplicaciones concretas:

- **Navegación:** en escritorio se muestra la barra completa de enlaces (`hidden lg:flex`); en pantallas menores se reemplaza por un **menú hamburguesa** que despliega las mismas opciones en un panel vertical.
- **Grillas adaptativas:** el catálogo muestra 1 columna en móvil, 2 en tablet y 3-4 en escritorio (`grid-cols-1 sm:grid-cols-2 lg:grid-cols-3`); las pantallas de administración y el carrito reorganizan sus columnas de igual forma.
- **Elementos condicionales:** componentes secundarios (como el selector de idioma) se ocultan en pantallas muy chicas para priorizar el contenido.

- **Tipografía y espaciado fluidos:** los tamaños de fuente y márgenes escalan por breakpoint para mantener la legibilidad.

7.4. Formularios y validaciones

Los formularios validan **en el cliente** (respuesta inmediata al usuario) y **siempre también en el servidor** (garantía real; ver 08 — Calidad).

Formulario	Validaciones en el cliente
Registro	Usuario obligatorio; contraseña de al menos 8 caracteres; confirmación coincidente; indicador de fortaleza de la contraseña; aceptación de términos obligatoria.
Inicio de sesión	Campos obligatorios; mensajes diferenciados para credenciales inválidas (401), cuenta bloqueada (423) y exceso de intentos (429).
Alta/edición de producto	Título y categoría obligatorios; precio y stock numéricos no negativos.
Roles	Nombre obligatorio; selección de permisos del catálogo.
Configuración IA	La URL debe ser válida y absoluta; botón de "probar conexión" antes de guardar.
Chat IA	No permite enviar mensajes vacíos ni cuando el asistente está fuera de línea.

Los errores del servidor se muestran junto al formulario con mensajes claros (no códigos técnicos), usando el componente `Field` (etiqueta, indicador de requerido, ayuda y error) y paneles de error.

7.5. Sistema de diseño (UI)

La interfaz sigue una estética **"cyber/arcade" oscura** coherente con la temática gamer:

- **Tema:** fondo oscuro (paleta `ink`), acentos neón (cian y magenta), efectos de brillo y animaciones sutiles.
- **Tipografías:** Orbitron para títulos (display), Rajdhani para texto y JetBrains Mono para datos técnicos.
- **Componentes reutilizables** (`src/components/ui/`): botones, paneles, chips, campos de formulario, modales, toasts, spinners, estados vacíos, visor de imágenes y encabezados de página/sección. Todas las pantallas se componen con estas piezas, lo que asegura consistencia visual.
- **Clases utilitarias propias** (`src/index.css`): `.panel`, `.btn-*`, `.input`, `.table-cyber`, definidas sobre Tailwind con `@layer components`.

Aspectos de accesibilidad implementados: atributo `lang="es"` en el documento, `aria-label` en botones de ícono (zoom, +/- del carrito, cerrar modal, menú móvil), cierre de modales con la tecla

Escape, `aria-pressed` en conmutadores y bloqueo del scroll de fondo cuando hay un modal abierto. Las limitaciones pendientes se detallan en [08 — Calidad y normativa](#).

7.6. Multi-idioma (i18n)

El sistema soporta **español, inglés y portugués** con una particularidad: las traducciones **no están en archivos del frontend, sino en la base de datos** (tablas `Languages` y `Translations`), y se sirven por la API.

Funcionamiento:

1. Al iniciar, la aplicación lee el idioma preferido guardado en el navegador (`localStorage`, clave `ventagamer.lang`) o usa español por defecto.
2. Descarga el diccionario del idioma desde `GET /api/i18n/translations/{código}` y lo registra en **i18next**.
3. Los componentes traducen con la función `t("clave")` — por ejemplo `t("nav.catalog")` → "Catálogo" / "Catalog" / "Catálogo".
4. El **selector de idioma** del encabezado permite cambiar en caliente: descarga el nuevo diccionario y re-renderiza la interfaz sin recargar.
5. En el registro, el idioma elegido se guarda como preferencia del usuario en su perfil.

Ventaja de este diseño: agregar o corregir traducciones no requiere recompilar ni redesplegar el frontend; basta modificar filas en la base de datos.

7.7. Estructura del código fuente

```

frontend/src/
├─ main.tsx           # Punto de entrada: monta React y carga el idioma
├─ App.tsx            # Definición de rutas + proveedor de TanStack Query
├─ index.css          # Tailwind + clases del sistema de diseño
├─ lib/               # Infraestructura transversal
│   ├─ api.ts         # Cliente Axios con interceptores JWT
│   ├─ i18n.ts        # Configuración de i18next (traducciones desde la API)
│   └─ productImage.ts # Imágenes de respaldo por categoría
├─ components/
│   ├─ Layout.tsx     # Encabezado, menú por permisos, pie, widget IA
│   ├─ LanguageSwitcher.tsx
│   └─ ui/            # Biblioteca de componentes reutilizables
├─ features/          # Lógica por dominio (API + tipos + componentes)
│   ├─ auth/          # Sesión: store Zustand, llamadas de login/registro
│   ├─ products/      # API y tipos del catálogo
│   ├─ cart/          # API y tipos de carrito/pedidos
│   ├─ admin/         # API de roles/usuarios
│   └─ ai/            # Chat IA: API, hook SignalR y widget
└─ routes/            # 17 páginas (una por ruta)
    
```


Calidad y Normativa

8. Calidad de software y normativa

Este documento reúne las prácticas de calidad aplicadas en VentaGamer (organización del código, validaciones, seguridad, usabilidad y accesibilidad) y el análisis de cumplimiento de la **Ley 25.326 de Protección de Datos Personales** de la República Argentina.

8.1. Código modular y buenas prácticas

Organización del código

- **Backend en capas (Clean Architecture):** cuatro proyectos con dependencias unidireccionales — el dominio no conoce la base de datos ni el framework web. Cada capa tiene una única responsabilidad (ver [04 — Arquitectura](#)).
- **Frontend por funcionalidades:** cada dominio (auth, catálogo, carrito, admin, IA) agrupa su API, tipos y componentes; la infraestructura compartida (cliente HTTP, i18n) vive en `lib/` y los componentes visuales reutilizables en `components/ui/`.
- **Controllers delgados / servicios con la lógica:** los controllers reciben la solicitud, delegan en un servicio a través de su interfaz y traducen el resultado a HTTP. La lógica de negocio es testeable sin servidor web.
- **Contratos explícitos:** la comunicación entre capas usa interfaces (`IProductService` , `ICartService` , ...) y DTOs; las entidades del dominio nunca viajan directamente al cliente.

Herramientas de calidad

Práctica	Herramienta	Qué garantiza
Tipado estático en el frontend	TypeScript (modo estricto)	Errores de tipo detectados al compilar, no en ejecución.
Tipado estático en el backend	C# / .NET 8	Ídem, más <i>nullability</i> verificada por el compilador.
Análisis de estilo	ESLint (+ plugins de React)	Estilo homogéneo y detección de patrones problemáticos.
Convenciones de nombres	Consistentes en todo el código	Entidades y servicios en inglés técnico; interfaz de usuario en español.
Versionado	Git	Historia de cambios revisable; el esquema de BD versionado como migraciones.

Principios de diseño aplicados

- **Separación de intereses:** presentación, lógica y datos en capas distintas.
- **Inversión de dependencias:** las capas internas definen interfaces; las externas las implementan.
- **No repetición (DRY):** la auditoría se implementa una sola vez como interceptor (no en cada servicio); el cliente HTTP y el manejo de errores del frontend están centralizados.
- **Inmutabilidad donde importa:** los ítems de un pedido son instantáneas (snapshot) que no cambian aunque cambie el catálogo.
- **Fallar de forma controlada:** excepciones de dominio específicas (`UserBlockedException` , stock insuficiente) traducidas a códigos HTTP claros.

8.2. Validaciones

La regla general del sistema es: **el cliente valida para ayudar al usuario; el servidor valida para proteger los datos**. Ninguna validación existe solo en el navegador.

Validación	Cliente (React)	Servidor (ASP.NET Core)
Campos obligatorios y formatos	Formularios con mensajes inmediatos	Modelos de request validados
Contraseña mínima (8 caracteres)	Registro, con indicador de fortaleza	Rechazo en <code>AuthService</code>
Usuario duplicado	—	HTTP 409 con verificación en BD
Stock disponible	Deshabilita botones sin stock	Verificación en carrito y otra vez dentro de la transacción de checkout
Cantidades positivas	Controles +/- acotados	Restricciones en la entidad y en BD (<code>CHECK</code>)
Permisos	Oculto opciones sin permiso	Políticas de autorización en cada endpoint
URL de configuración IA	Validación de formato	<code>Uri.TryCreate</code> antes de persistir

8.3. Seguridad

Mecanismo	Implementación	Contra qué protege
Hash de contraseñas PBKDF2	Hasher de ASP.NET Identity con salt individual; nunca se almacena texto plano.	Robo de credenciales ante una filtración de la BD.
Tokens JWT firmados	HMAC-SHA256, vigencia 60 minutos, validación de emisor/audiencia/vencimiento.	Suplantación de identidad y manipulación del token.
Bloqueo de cuenta	3 intentos fallidos consecutivos → cuenta bloqueada (HTTP 423); desbloqueo solo por administrador.	Adivinación de contraseñas dirigida a una cuenta.
Rate limiting	60 solicitudes/min por IP global; 5/min en login y registro; 10 mensajes/min en el chat IA.	Fuerza bruta y abuso del servicio.
Autorización granular	12 permisos verificados por políticas en el servidor, en cada solicitud.	Escalamiento de privilegios manipulando la interfaz.
Consultas parametrizadas	Todo el acceso a datos pasa por EF Core (LINQ → SQL parametrizado).	Inyección SQL.
Protección XSS	React escapa por defecto todo el contenido interpolado.	Inyección de scripts en la interfaz.
CORS restringido	Solo los orígenes del frontend configurados pueden llamar a la API desde el navegador.	Solicitudes desde sitios no autorizados.
Transacciones	El checkout es atómico: o se completa todo o no se hace nada.	Estados inconsistentes (stock descontado sin pedido).
Auditoría automática	Interceptor de EF Core registra todo cambio en entidades sensibles, con usuario y fecha.	Operaciones sin rastro; facilita el análisis posterior de incidentes.
Configuración fuera del código	Claves (JWT, contraseña de BD) inyectadas por variables de entorno (<code>.env</code>).	Exposición de secretos en el repositorio.

8.4. Usabilidad

- **Retroalimentación inmediata:** indicadores de carga (spinners), avisos de éxito (toasts), mensajes de error específicos y en lenguaje claro.
- **Estados vacíos cuidados:** cuando no hay resultados, carrito vacío o faltan permisos, se muestra una pantalla explicativa con la acción sugerida, nunca un espacio en blanco.
- **Prevención de errores:** botones deshabilitados cuando la acción no es válida (sin stock, formulario incompleto, asistente fuera de línea).

- **Consistencia:** todas las pantallas usan la misma biblioteca de componentes, por lo que botones, tablas y formularios se ven y comportan igual en todo el sistema.
- **Navegación clara:** menú según permisos (el usuario no ve opciones que no puede usar), página 404 propia, ayuda con preguntas frecuentes.
- **Usuarios demo:** la pantalla de login ofrece credenciales de demostración por rol, facilitando la evaluación del sistema.

8.5. Accesibilidad

Implementado:

- Idioma del documento declarado (`lang="es"`).
- `aria-label` en botones de solo ícono (ampliar imagen, +/- del carrito, cerrar modal, menú móvil).
- Cierre de modales y visor de imágenes con la tecla **Escape**; bloqueo del scroll de fondo.
- `aria-pressed` en conmutadores; `aria-hidden` en elementos decorativos.
- Contraste alto por diseño (texto claro sobre fondo oscuro).

Limitaciones conocidas (mejoras futuras): no hay enlaces de salto ("saltar al contenido"), el foco no queda atrapado dentro de los modales abiertos, algunas animaciones intensas no respetan aún la preferencia de "reducir movimiento" del sistema, y la navegación completa por teclado no está verificada en todas las pantallas. Se documentan explícitamente como parte del análisis de calidad.

8.6. Ley 25.326 — Protección de Datos Personales

La **Ley 25.326** regula el tratamiento de datos personales en la Argentina: exige que los datos se recolecten con un fin determinado y consentimiento del titular (arts. 4-6), que se garantice su **seguridad y confidencialidad** (arts. 9-10), y que el titular pueda ejercer los derechos de **acceso, rectificación y supresión** (arts. 14-16).

Datos personales que trata el sistema

VentaGamer aplica el principio de **minimización**: recolecta únicamente los datos indispensables.

Dato	Finalidad	Base de licitud
Nombre de usuario	Identificación de la cuenta	Consentimiento al registrarse
Contraseña (solo hash PBKDF2)	Autenticación	Consentimiento al registrarse
Idioma preferido	Personalización de la interfaz	Consentimiento al registrarse
Historial de compras	Gestión de pedidos y comprobantes	Ejecución de la relación de consumo
Fecha de último acceso e intentos fallidos	Seguridad de la cuenta	Interés legítimo (seguridad)
Conversaciones con el asistente IA	Continuidad de la conversación	Consentimiento (uso voluntario del asistente)

Cabe destacar que el sistema **no** recolecta datos sensibles (art. 7): no se piden nombre real, dirección, teléfono ni datos financieros (no hay pasarela de pagos en esta versión).

Cómo cumple el sistema cada exigencia

Exigencia de la ley	Implementación en VentaGamer
Consentimiento informado (arts. 5-6)	El formulario de registro exige aceptar los términos y el aviso de privacidad antes de crear la cuenta.
Calidad y finalidad de los datos (art. 4)	Solo se piden los datos necesarios para operar; no se usan con fines distintos ni se ceden a terceros.
Seguridad de los datos (art. 9)	Contraseñas con hash, autenticación JWT, autorización granular, rate limiting, CORS restringido y transacciones (detallado en 8.3).
Deber de confidencialidad (art. 10)	Acceso a datos de otros usuarios restringido a perfiles administrativos con permisos específicos; toda consulta administrativa queda auditada.
Derecho de acceso (art. 14)	El usuario puede consultar sus datos (perfil, pedidos, conversaciones) desde su sesión.
Derecho de rectificación y supresión (art. 16)	El usuario puede eliminar sus conversaciones con el asistente; la administración puede corregir o dar de baja cuentas. La bitácora conserva los eventos de forma disociada (si se elimina el usuario, sus registros de auditoría quedan sin referencia personal — <code>SET NULL</code>).
Procesamiento local de IA	Las consultas al asistente se procesan en un servidor propio (Ollama, local): los datos de los usuarios no se envían a servicios de IA de terceros.

Aviso de privacidad

El sistema incluye la aceptación de términos en el registro. El siguiente texto modelo de **aviso de privacidad** acompaña al sistema y resume la política de tratamiento de datos:

Aviso de privacidad — VentaGamer. En cumplimiento de la Ley 25.326 de Protección de Datos Personales, se informa que los datos recabados al registrarse (nombre de usuario, contraseña e idioma preferido) y los generados por el uso del sistema (historial de compras y conversaciones con el asistente virtual) serán utilizados exclusivamente para operar la plataforma: autenticar su cuenta, gestionar sus compras y personalizar su experiencia. Las contraseñas se almacenan cifradas de forma irreversible y ningún dato se cede a terceros. Usted puede ejercer en todo momento sus derechos de acceso, rectificación y supresión de datos contactando al administrador del sistema. El titular de los datos podrá, asimismo, presentar reclamos ante la Agencia de Acceso a la Información Pública, órgano de control de la Ley 25.326.

8.7. Trazabilidad: la bitácora como mecanismo de control

La bitácora de auditoría (ver CU-08 en 03 — Casos de uso) es también un instrumento de cumplimiento: permite demostrar **quién** realizó cada operación sobre los datos y **cuándo**. Al implementarse como interceptor a nivel del acceso a datos, ninguna modificación de productos, usuarios, roles o pedidos puede eludir el registro, lo que respalda tanto la seguridad interna como la rendición de cuentas exigible a un responsable de base de datos según la normativa.

Chatbot con IA

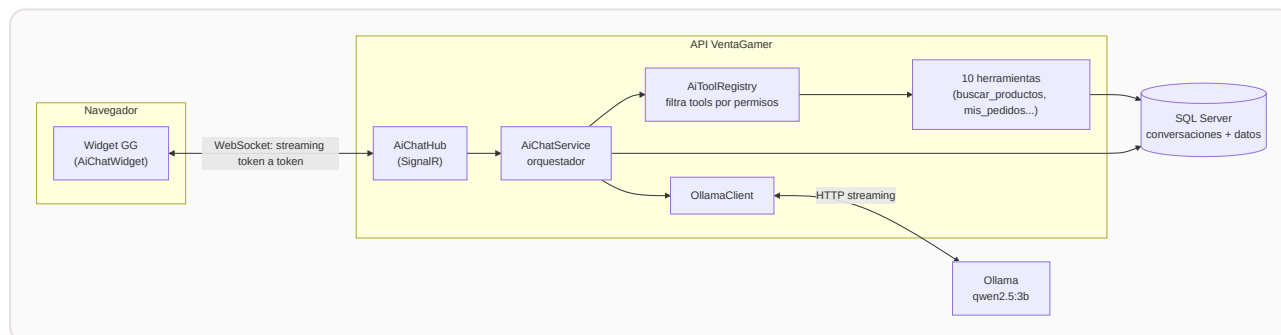
9. Chatbot con IA: el asistente "GG"

VentaGamer implementa la funcionalidad opcional de nivel avanzado de la consigna: un **chatbot integrado con inteligencia artificial**. El asistente se llama **GG** (*Game Guide*) y permite a los usuarios consultar el sistema en lenguaje natural: buscar productos, verificar stock, revisar pedidos y —para administradores— consultar indicadores de ventas.

Lo distintivo de la implementación es que el asistente **no inventa datos**: consulta la base de datos real mediante un mecanismo de *tool calling* (invocación de herramientas), y solo puede acceder a la información que los **permisos del usuario** que consulta le habilitan.

9.1. Arquitectura general

El motor de lenguaje es **Ollama**, un servidor de modelos de IA que corre **localmente** (como contenedor Docker propio), sin depender de servicios en la nube: ninguna conversación sale de la infraestructura del sistema. El modelo configurado por defecto es **qwen2.5:3b**, un modelo compacto (~2 GB) con soporte de invocación de herramientas.

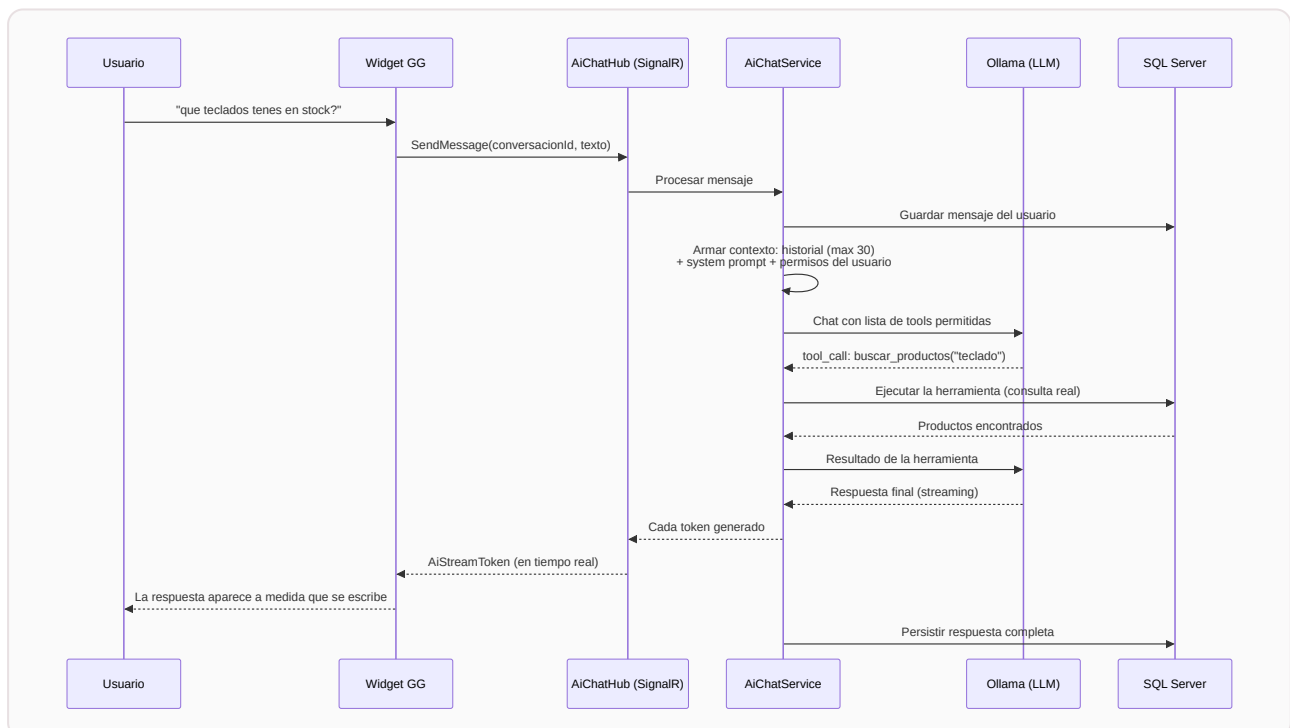


Componentes (todos en `backend/src/VentaGamer.Infraestructura/Ai/` salvo indicación):

Componente	Rol
<code>AiChatWidget</code> (frontend)	Ventana de chat flotante, disponible en toda la aplicación para usuarios autenticados. Renderiza las respuestas con formato (Markdown).
<code>AiChatHub</code> (API, SignalR)	Canal WebSocket: recibe el mensaje del usuario y transmite la respuesta token a token.
<code>AiChatService</code>	Orquestador: arma el contexto, ejecuta el ciclo de tool calling y persiste la conversación.
<code>AiToolRegistry</code>	Decide qué herramientas están disponibles para este usuario , según sus permisos.
<code>OllamaClient</code>	Cliente HTTP hacia Ollama; lee URL y modelo desde la configuración en BD en cada solicitud.
Tablas <code>AiConversations</code> / <code>AiMessages</code>	Persistencia del historial: las conversaciones se pueden retomar.

9.2. El ciclo de tool calling

El modelo de lenguaje no accede a la base de datos directamente. En cambio, el sistema le describe un conjunto de **herramientas** (funciones con nombre, descripción y parámetros) y el modelo decide cuándo invocarlas. El backend ejecuta la herramienta, le devuelve el resultado y el modelo continúa hasta poder responder.



Salvaguardas del ciclo:

- **Máximo 8 rondas** de herramientas por mensaje (evita bucles infinitos).
- **Historial acotado** a los últimos 30 mensajes (controla el tamaño del contexto).
- **Throttling**: máximo 10 mensajes por minuto por conexión.
- Si Ollama está fuera de línea, el widget lo indica claramente y deshabilita el envío.

9.3. Las herramientas y sus permisos

Las 10 herramientas disponibles están en `backend/src/VentaGamer.Infrastructure/Ai/Tools/`. El registro (`AiToolRegistry`) **filtra según los permisos del usuario**: el modelo ni siquiera "ve" las herramientas que el usuario no puede usar, por lo que es imposible que un cliente consulte datos administrativos a través del chat.

Herramienta	Qué hace	Permiso requerido
<code>buscar_productos</code>	Busca en el catálogo por título o categoría (máximo 10 resultados).	Ninguno (catálogo público)
<code>consultar_stock</code>	Stock actual de un producto por su ID.	Ninguno
<code>listar_categorias</code>	Categorías de productos activas.	Ninguno
<code>listar_capacidades</code>	Meta-herramienta: informa al modelo qué puede hacer para este usuario.	Ninguno
<code>mis_pedidos</code>	Pedidos del usuario que consulta.	<code>orders.read.own</code>
<code>detalle_pedido</code>	Detalle de un pedido (propio; o de cualquiera con permiso ampliado).	<code>orders.read.own</code> / <code>orders.read.all</code>
<code>kpis_admin</code>	Indicadores globales: ventas, facturación, usuarios.	<code>orders.read.all</code>
<code>top_productos_vendidos</code>	Ranking de productos más vendidos.	<code>orders.read.all</code>
<code>pedidos_recientes</code>	Últimos pedidos del sistema.	<code>orders.read.all</code>

Ejemplos de interacción:

- **Cliente**: "¿tenés auriculares por menos de \$120?" → el modelo invoca `buscar_productos("auriculares")`, filtra por precio y responde con los productos reales, incluyendo IDs y stock.
- **Cliente**: "¿cuándo hice mi última compra?" → `mis_pedidos` → responde con el número de orden y la fecha reales.
- **Administrador**: "¿cuál fue el producto más vendido este mes?" → `top_productos_vendidos` → responde con el ranking real.

9.4. Personalidad y reglas del asistente

El comportamiento del asistente se define en un *system prompt* (en `OllamaOptions`, `backend/src/VentaGamer.Application/Ai/OllamaOptions.cs`) que establece:

- **Identidad:** es "GG", asistente de VentaGamer, con tono cordial y temática gamer moderada; responde en español.
- **Regla de veracidad:** nunca inventa precios, stock ni fechas — todo dato debe provenir de una herramienta.
- **Filosofía "nunca un no plano":** antes de responder "no puedo", intenta otras herramientas o combinaciones; si el usuario no tiene permisos para algo, se lo explica indicando qué permiso falta.
- **Formato:** usa Markdown (negritas para precios y nombres, tablas para comparaciones) e incluye los IDs de productos para que el usuario pueda seguir preguntando.

9.5. Configuración en tiempo de ejecución

La pantalla `/admin/ai` permite administrar el asistente **sin reiniciar el sistema**:

- **URL del servidor Ollama:** puede apuntar al contenedor local o a otra máquina de la red.
- **Modelo:** seleccionable entre los instalados en Ollama (por ejemplo, cambiar a un modelo más grande si el hardware lo permite).
- **Probar conexión** antes de guardar.

Los valores se persisten en la tabla `SystemSettings` (claves `Ai:BaseUrl` y `Ai:Model`) y el cliente los relee en cada solicitud, con la configuración estática como respaldo. El estado (en línea / fuera de línea) y los modelos disponibles se muestran en la misma pantalla.

9.6. Privacidad

Las conversaciones se procesan **íntegramente en infraestructura propia**: Ollama corre como contenedor local y no se envía ningún dato a servicios de IA de terceros. Cada usuario solo accede a sus propias conversaciones, puede eliminarlas cuando lo desee (borrado en cascada de sus mensajes) y el asistente opera bajo los mismos permisos que el usuario tiene en el resto del sistema.

Backup y Recuperación

10. Estrategia de backup y recuperación

VentaGamer implementa la funcionalidad opcional de nivel avanzado de respaldo de información: generación de **copias de seguridad de la base de datos** desde la propia aplicación, historial de respaldos y **verificación de integridad** de los datos críticos. Este documento responde las tres preguntas que exige la consigna: qué datos se respaldan, con qué frecuencia, y cómo se recupera la información ante fallos.

10.1. Qué datos se respaldan

El backup abarca la **base de datos completa** (`VentaGamer` en SQL Server), lo que incluye:

Área	Tablas	Criticidad
Seguridad	Users, Roles, Permissions, RolePermissions, RoleHierarchies	Alta — cuentas y esquema de acceso
Ventas	Orders, OrderItems	Máxima — registro contable del negocio, irre recuperable si se pierde
Catálogo	Products	Alta — aunque puede reconstruirse, con costo
Operación	Carts, CartItems	Baja — datos transitorios
Auditoría	AuditLogs	Alta — trazabilidad y cumplimiento
Configuración	SystemSettings, Languages, Translations	Media — reconstruible desde el seed
Chatbot	AiConversations, AiMessages	Media — datos personales de los usuarios

Se respalda la base completa (y no tablas sueltas) porque garantiza la **consistencia referencial**: un respaldo parcial podría restaurar pedidos que referencian usuarios o productos inexistentes.

10.2. Cómo se genera el backup

Desde la pantalla `/maintenance` (permiso `backup.manage`), el usuario técnico ejecuta la creación del respaldo. El backend (`MaintenanceService`) lanza un comando nativo de SQL Server:

```

BACKUP DATABASE [VentaGamer]
TO DISK = '/var/opt/mssql/data/backups/VentaGamer_{fecha}.bak'
WITH COMPRESSION;
    
```

Puntos clave:

- El archivo `.bak` es un **backup completo y comprimido**, el formato estándar de SQL Server.
- Se escribe dentro del **volumen Docker** `sqldata`, que persiste aunque el contenedor se destruya y recree.
- La pantalla muestra el **historial de respaldos** (fecha, tamaño, archivo), consultado del catálogo interno del motor (`msdb.dbo.backupset`), por lo que refleja también los backups realizados por fuera de la aplicación.
- Toda la operatoria queda auditada en la bitácora (quién generó cada backup y cuándo).

10.3. Política de frecuencia propuesta

Para un despliegue productivo de un comercio de este tipo se propone el siguiente esquema, escalonado según la criticidad de los datos:

Tipo de respaldo	Frecuencia	Justificación
Backup completo (el implementado)	Diario, fuera del horario pico (por ejemplo, 3:00)	Restaura el sistema entero a un punto conocido.
Backup diferencial	Cada 6 horas	Reduce la pérdida máxima de datos a horas, con archivos chicos.
Backup del log de transacciones	Cada 15-30 minutos	Permite recuperación a un punto en el tiempo (por ejemplo, justo antes de un error).
Copia fuera del servidor	Diaria, tras el backup completo	Un respaldo que vive en el mismo disco que la base no protege contra la falla de ese disco: debe copiarse a otro equipo o almacenamiento.
Prueba de restauración	Mensual	Un backup solo es válido si se demostró que restaura: se verifica en un entorno de prueba.

Retención sugerida: 7 backups diarios, 4 semanales y 12 mensuales, eliminando los anteriores.

En el entorno de demostración del trabajo práctico, el backup se dispara **a demanda** desde la interfaz; la programación automática se resolvería con un programador de tareas (cron en el host o SQL Server Agent) invocando el mismo comando.

10.4. Procedimiento de recuperación ante fallos

La aplicación genera y lista los backups; la **restauración es un procedimiento manual y deliberado** del administrador técnico. Es una decisión de diseño: restaurar una base sobrescribe todos los datos actuales, por lo que no conviene exponerlo como un botón en la interfaz web.

Procedimiento documentado (con el sistema en Docker):

```

# 1. Detener la API para que no haya conexiones activas contra la base
docker compose stop backend

# 2. Listar los backups disponibles dentro del contenedor de SQL Server
docker exec ventagamer-sql ls /var/opt/mssql/data/backups/

# 3. Restaurar el backup elegido (sobrescribe la base actual)
docker exec ventagamer-sql /opt/mssql-tools18/bin/sqlcmd \
    -S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C \
    -Q "RESTORE DATABASE [VentaGamer] \
        FROM DISK = '/var/opt/mssql/data/backups/VentaGamer_2026xxx.bak' \
        WITH REPLACE;"

# 4. Volver a levantar la API (aplica migraciones pendientes si las hubiera)
docker compose start backend
    
```

Escenarios de falla cubiertos:

Escenario	Recuperación
Error humano (borrado o modificación masiva incorrecta)	Restaurar el último backup completo previo al error.
Corrupción de la base	Restaurar backup; el esquema se revalida con las migraciones al arrancar la API.
Pérdida del contenedor o del volumen	Recrear los contenedores con <code>docker compose up</code> ; restaurar el <code>.bak</code> desde la copia externa.
Migración de servidor	Copiar el <code>.bak</code> al nuevo host, levantar el stack y restaurar.

10.5. Verificación de integridad (HMAC-SHA256)

Complementando el respaldo, el sistema ofrece una **verificación de integridad** de los datos críticos (permiso `integrity.check`, misma pantalla de mantenimiento):

- El servicio recorre las tablas **Users** y **Orders** y calcula para cada registro una firma **HMAC-SHA256** sobre una representación canónica de sus campos (por ejemplo, para un pedido: número, usuario, total y fecha).
- HMAC combina la función de hash SHA-256 con una **clave secreta**: sin la clave, no es posible generar firmas válidas. Esto permite detectar modificaciones hechas por fuera de la aplicación (por ejemplo, un UPDATE directo en la base alterando el total de un pedido).
- El resultado es un reporte por tabla con las firmas calculadas y las anomalías detectadas.

Alcance actual y mejora futura: en esta versión el mecanismo calcula y demuestra las firmas en el momento de la verificación. El paso siguiente natural es **almacenar la firma junto a cada registro** al crearlo, de modo que la verificación compare firma almacenada contra firma recalculada y detecte cualquier alteración posterior de manera concluyente.

10.6. Otras capas de protección de datos

La estrategia de respaldo se apoya en decisiones de diseño que reducen la probabilidad de necesitarla:

- **Baja lógica de productos:** eliminar un producto no borra datos, solo lo desactiva; el historial de ventas queda intacto.
- **Snapshot en pedidos:** los ítems de un pedido copian título y precio al momento de la compra; el registro contable no depende del estado actual del catálogo.
- **Transacciones en checkout:** imposible llegar a estados intermedios inconsistentes.
- **Políticas de borrado cuidadas:** los pedidos no se borran en cascada; la bitácora sobrevive a la eliminación de usuarios (detalle en [05 — Base de datos](#)).
- **Volúmenes Docker persistentes:** los datos de SQL Server (`sqldata`) y los modelos de IA (`ollamadata`) sobreviven a la recreación de los contenedores.

Ejecución del Proyecto

11. Cómo ejecutar el proyecto

Este documento explica cómo poner en funcionamiento VentaGamer. La forma recomendada es con **Docker**, que levanta el sistema completo (frontend, backend, base de datos y motor de IA) con un solo comando y sin instalar ninguna otra dependencia. Al final se describe también el modo de desarrollo local.

11.1. Requisitos

Requisito	Detalle
Docker con Compose v2	Único requisito para el modo recomendado. En Linux: <code>docker.io</code> + <code>docker-compose-v2</code> ; en Windows/macOS: Docker Desktop.
Espacio en disco	~5 GB libres (imágenes de los contenedores + modelo de IA de ~2 GB).
Puertos libres	8080 (web), 5050 (API), 1434 (SQL Server), 11434 (Ollama).

11.2. Modo recomendado: todo con Docker

El repositorio incluye dos scripts en la raíz que automatizan la puesta en marcha:

Paso 1 — Preparación (una sola vez)

```
cd VentaGamer-Modern
sudo ./setup-docker.sh
```

Este script habilita el servicio de Docker, agrega el usuario actual al grupo `docker` (para no necesitar `sudo` en adelante) y prepara el archivo de configuración `.env` con los valores de desarrollo, incluido el modelo de IA por defecto (`qwen2.5:3b`). Tras ejecutarlo, aplicar el nuevo grupo con `newgrp docker` (o cerrando y reabriendo la sesión).

Paso 2 — Levantar el sistema

```
./levantar.sh
```

El script ejecuta `docker compose up -d --build --wait`, que:

1. Construye las imágenes del backend (compilación `.NET`) y del frontend (build de Vite + nginx).

- Levanta SQL Server y Ollama, esperando a que sus verificaciones de salud den positivo.
- Levanta la API, que **aplica las migraciones y puebla los datos iniciales automáticamente** (roles, permisos, usuarios demo, 20 productos, traducciones).
- Descarga el modelo de IA la primera vez (~2 GB; puede tardar varios minutos).

La primera ejecución completa puede llevar varios minutos (descarga de imágenes, compilación y modelo). Las siguientes son casi inmediatas.

Resultado

Servicio	URL
Aplicación web	http://localhost:8080
API (Swagger, solo en modo desarrollo)	http://localhost:5050/swagger
Verificación de salud de la API	http://localhost:5050/api/health
SQL Server	localhost,1434 (usuario sa)
Ollama	http://localhost:11434

Usuarios de demostración

El sistema se inicializa con cuentas de prueba que cubren todos los perfiles:

Usuario	Contraseña	Rol	Qué permite probar
cliente	Cliente123!	Cliente (User)	Catálogo, carrito, checkout, comprobante PDF, chat IA
juan	Juan123!	Cliente (User)	Segundo cliente (para ver pedidos de varios usuarios)
admin	Admin123!	Administrador	ABM de productos, usuarios, roles y permisos, configuración IA, todas las compras
webmaster	WebMaster123!	WebMaster	Bitácora de auditoría, backups, verificación de integridad
tester	Tester123!	Tester	Perfil mixto de consulta (QA)

Comandos útiles

```

docker compose ps          # Estado de los 4 contenedores
docker compose logs -f     # Logs en vivo (Ctrl+C para salir)
docker compose logs backend # Logs solo de la API
docker compose down        # Detener todo (los datos persisten en los volúmenes)
docker compose down -v     # Detener y BORRAR los datos (reinicio de cero)
  
```


11.3. Recorrido de prueba sugerido

Para verificar el sistema completo en unos minutos:

1. **Como visitante:** abrir <http://localhost:8080>, navegar el catálogo, buscar y filtrar por categoría (sin iniciar sesión).
2. **Como cliente** (`cliente` / `Cliente123!`): agregar productos al carrito, modificar cantidades, confirmar la compra y **descargar el comprobante PDF** desde el detalle del pedido. Probar el asistente **GG** (botón flotante): preguntar por ejemplo "¿qué consolas tenés en stock?".
3. **Como administrador** (`admin` / `Admin123!`): crear o editar un producto en `/admin/products` ; revisar todas las compras en `/admin/orders` ; ver roles y permisos en `/admin` ; verificar el estado del asistente en `/admin/ai` .
4. **Como webmaster** (`webmaster` / `WebMaster123!`): revisar en `/audit` la bitácora (deberían aparecer las operaciones recién realizadas) y generar un backup en `/maintenance` .
5. **Responsive:** achicar la ventana o abrir desde un móvil — el menú pasa a hamburguesa y las grillas se reorganizan.
6. **Multi-idioma:** cambiar el idioma con el selector del encabezado (es/en/pt).

11.4. Modo alternativo: desarrollo local

Para trabajar sobre el código con recarga en caliente se puede ejecutar cada pieza por separado. Requiere: **.NET 8 SDK**, **Node.js 20+** y Docker (solo para la base y Ollama).

```
# 1. Base de datos y Ollama en Docker
docker compose up -d sqlserver ollama

# 2. Backend (puerto 5050)
cd backend
dotnet restore
dotnet run --project src/VentaGamer.Api

# 3. Frontend (puerto 5173, en otra terminal)
cd frontend
npm install
npm run dev
```

En este modo la aplicación queda en <http://localhost:5173>; el servidor de Vite reenvía `/api` y `/hubs` al backend en el puerto 5050. Swagger queda disponible en <http://localhost:5050/swagger>.

11.5. Solución de problemas frecuentes

Problema	Causa probable	Solución
<code>permission denied</code> <code>... docker.sock</code>	El usuario no pertenece al grupo <code>docker</code> .	Ejecutar <code>sudo ./setup-docker.sh</code> y luego <code>newgrp docker</code> (o reiniciar la sesión).
Un puerto ya está en uso	Otro servicio ocupa 8080/5050/1434/11434.	Liberar el puerto o cambiar el mapeo en <code>docker-compose.yml</code> .
El backend se reinicia al arrancar	SQL Server todavía estaba iniciándose.	Esperar: las dependencias con healthcheck lo reintentan solos. Ver <code>docker compose logs backend</code> .
El asistente GG figura "offline"	El modelo aún se está descargando, u Ollama no está corriendo.	Verificar con <code>docker compose logs ollama</code> ; el script <code>levantar.sh</code> descarga el modelo automáticamente.
La web carga pero las llamadas fallan	El backend no está sano.	<code>docker compose ps</code> debe mostrar los 4 contenedores "healthy"; revisar logs de la API.
Se quiere reiniciar los datos de cero	—	<code>docker compose down -v &&</code> <code>./levantar.sh</code> (borra la base y vuelve a sembrar los datos demo).